

Final Year Project Report

Full Unit - Final Report

Comparison of Machine Learning Algorithms

Tayib Sheikh

A report submitted in part fulfilment of the degree of

BSc (Hons) in Computer Science

Supervisor: Xu Feng



Department of Computer Science
Royal Holloway, University of London

April 15, 2026

Declaration

This report has been prepared on the basis of my own work. Where other published and unpublished source materials have been used, these have been acknowledged.

Word Count: 15,000 approx.

Student Name: Tayib Sheikh

Date of Submission: 15/04/2026

Signature: Tayib.

Table of Contents

Abstract	7
Project Specification	8
1 Introduction	9
1.1 Project Background and Motivation	9
1.2 Problem Statement	10
1.3 Aims and Objectives	10
1.4 Research Questions	11
1.5 Project Contributions	12
1.6 Report Structure	12
2 Background and Literature Review	13
2.1 Supervised Learning and Generalisation	13
2.2 Class Imbalance in Classification	14
2.3 Evaluation Metrics for Imbalanced Problems	15
2.4 k-Nearest Neighbours	15
2.5 Decision Trees	16
2.6 Support Vector Machines	17
2.7 Related Work	18
2.8 Critical Summary of the Literature	19
3 Software Design and Implementation	21
3.1 Overall System Architecture	21
3.2 Repository Structure	22
3.3 Design Goals and Requirements	22
3.4 Algorithm Implementations	23
3.5 Data Loading and Preprocessing Components	25

3.6	Evaluation Pipeline	25
3.7	Hyperparameter Search Framework	25
3.8	Visualisation and Results Generation	26
3.9	Testing and Validation	26
3.10	Reproducibility and Documentation	27
4	Experimental Methodology	28
4.1	Datasets	28
4.2	Preprocessing Strategy	29
4.3	Experimental Protocol	30
4.4	Hyperparameter Tuning Strategy	31
4.5	Baseline Comparison with scikit-learn	31
4.6	Performance Metrics	32
4.7	Threats to Validity	32
5	Results	34
5.1	Implementation Validation Against scikit-learn	34
5.2	Standard Benchmark Results	35
5.3	Bitcoin Heist Under the Strict Report Pipeline	37
5.4	Paired Prediction-Level Comparison	38
5.5	Threshold Behaviour on Strict Bitcoin	39
5.6	Effect of Methodological Corrections on Bitcoin Heist	40
5.7	Summary of Main Findings	41
6	Discussion	42
6.1	Interpretation of the Results	42
6.2	From-Scratch Models Versus Library Baselines	43
6.3	Impact of Class Imbalance and Metric Choice	43
6.4	Dataset-Specific Behaviour	44
6.5	Technical Lessons from Development	46
6.6	Comparison with the Literature	47
6.7	Overall Discussion	47
7	Project Evaluation and Reflection	48

7.1 Assessment Against Original Objectives 48

7.2 What Went Well 48

7.3 What Went Wrong 49

7.4 Lessons Learned 50

7.5 Limitations 50

7.6 Future Work 50

8 Conclusion 52

8.1 Final Conclusion 52

Bibliography 54

A Video Demonstration Link 57

B User Manual 58

C Installation Manual 59

List of Figures

3.1	High-level architecture of the experimentation framework.	21
5.1	Final non-Bitcoin comparison between the custom implementations and the corresponding scikit-learn baselines.	34
5.2	Threshold trade-off for the strict Bitcoin custom Decision Tree. The selected threshold is marked explicitly in the figure.	39
5.3	Threshold trade-off for the strict Bitcoin custom SVM. The selected threshold is marked explicitly in the figure. Score magnitudes should be interpreted within-model rather than compared numerically across model families.	39
5.4	Bitcoin performance under the earlier easy random-split evaluation and the newer strict temporal-holdout evaluation.	40

List of Tables

4.1	Core datasets used in the final experimental framework.	28
4.2	Dataset-specific preprocessing steps used in the experimental pipeline.	30
4.3	Metric emphasis across datasets in the final evaluation design.	32
5.1	Final F1-scores for the custom implementations and the corresponding scikit-learn baselines. Bitcoin KNN is omitted because it was not part of the final strict Bitcoin report pipeline.	34
5.2	Final Iris holdout results.	35
5.3	Final Breast Cancer holdout results. AP denotes average precision.	36
5.4	Final Bank Marketing holdout results. AP denotes average precision.	36
5.5	Best strict Bitcoin configurations selected during grouped cross-validation on pre-2017 data.	37
5.6	Strict Bitcoin holdout results on the unseen 2017 evaluation set. AP denotes average precision.	37
5.7	Paired custom-versus-sklearn comparison on the shared final test sets. “Custom only correct” counts cases where the custom model is correct and sklearn is wrong, while “sklearn only correct” counts the reverse.	38
5.8	McNemar-style paired comparison of custom and sklearn hard predictions on the shared final test sets. b denotes cases where the custom model is correct and sklearn is wrong; c denotes the reverse.	38

Abstract

This report presents a comparative study of three classical supervised machine learning algorithms for tabular classification, namely k-Nearest Neighbours, Decision Trees, and Support Vector Machines. The project implements these models from scratch in Python within a unified and reproducible experimental framework supporting preprocessing, cross-validation, hyperparameter tuning, metric computation, and comparison against scikit-learn baselines. The empirical evaluation uses four datasets spanning balanced and imbalanced settings: Iris, Breast Cancer Wisconsin Diagnostic, Bank Marketing, and Bitcoin Heist. Results on the standard benchmark datasets show that the custom implementations achieve broad agreement with trusted library models, especially for k-Nearest Neighbours and Decision Trees, supporting their technical correctness. The experiments also show that model performance varies substantially with dataset structure, class imbalance, and feature representation, with no single algorithm dominating all tasks. The most important finding arises from the Bitcoin Heist case study, where a stricter temporally-aware evaluation pipeline produces far weaker but more credible results than earlier optimistic random-split experiments. This demonstrates that trustworthy evaluation can be as important as algorithm choice itself. Overall, the project contributes both a set of validated from-scratch implementations and a methodological case for rigorous evaluation in imbalanced tabular machine learning.

Project Specification

The aim of this project was to design, implement, and evaluate a compact machine learning experimentation framework for comparing three classical supervised classification algorithms: k-Nearest Neighbours, Decision Trees, and Support Vector Machines. The project was intended to combine algorithmic understanding, software engineering, and empirical analysis through the development of from-scratch model implementations and their application to tabular datasets of varying complexity and practical importance.

The project objectives were as follows. First, to implement the selected algorithms from first principles in Python using a consistent `fit/predict` style interface. Second, to build a reusable experimental pipeline supporting data loading, preprocessing, cross-validation, hyperparameter tuning, metric computation, and results logging. Third, to evaluate the custom models on multiple datasets, including balanced and imbalanced classification problems drawn from scientific, commercial, and cybersecurity contexts. Fourth, to compare the custom implementations against scikit-learn baselines under matched conditions in order to assess implementation validity. Fifth, to analyse how methodological choices, particularly metric selection, class imbalance, and evaluation protocol, affect the conclusions that can be drawn from comparative machine learning experiments.

The main deliverables of the project were a from-scratch Python codebase, a reproducible experimentation framework, a set of generated result files and visualisations, and a final report documenting the theoretical background, implementation process, experimental methodology, empirical findings, and critical evaluation of the work. The project therefore had both a technical goal, namely the development of working and validated implementations, and an analytical goal, namely to draw principled conclusions about classifier behaviour on tabular problems where minority class performance and evaluation validity are central.

Chapter 1: Introduction

1.1 Project Background and Motivation

Machine learning is widely used in domains such as finance, healthcare, and cybersecurity. Although deep learning dominates many high-profile applications, a large number of practically important prediction tasks still arise in the form of structured, tabular data. Banks store customer and campaign records in tables, hospitals record diagnostic variables in tabular form, and blockchain activity can be represented through engineered numerical features. In such settings, the challenge is not to process images or language, but to make reliable classification decisions from structured attributes.

Many of these tasks are also *imbalanced* and *cost-sensitive*. The cases of greatest practical interest are often the minority class. A customer likely to subscribe to a financial product is rare relative to those who are not. A malignant tumour is less common than a benign one, yet missing it is far more serious. A ransomware-related cryptocurrency address is rare compared with benign blockchain activity, but failing to detect it may allow harmful behaviour to continue. In such settings, overall accuracy alone can be misleading, because a classifier may appear successful while still performing poorly on the minority cases that matter most [19, 14, 21].

This project therefore focuses on a more meaningful question than generic benchmarking. Rather than asking only which model achieves the highest score, it asks how classical machine learning algorithms should be implemented, validated, and compared so that they can be trusted on imbalanced, cost-sensitive tabular problems. This is important theoretically because it concerns inductive bias, data geometry, and generalisation. It is important practically because reported performance may reflect genuine predictive ability, or instead reflect artefacts of metric choice, preprocessing, or flawed evaluation design.

The study centres on three classical supervised learning families: k-Nearest Neighbours (k-NN), Decision Trees, and Support Vector Machines (SVMs). These embody different inductive biases. k-NN predicts through local similarity in feature space [4]. Decision Trees learn recursive feature-based splits [5, 6]. SVMs learn margin-maximising decision boundaries through optimisation [7]. Following Mitchell [2], this project treats these differences as differences in inductive bias rather than simply differences in implementation style.

To study these behaviours, the project implements all three algorithm families from scratch and evaluates them within a unified and reproducible framework. The main datasets are Bank Marketing [25, 26], Breast Cancer, and Bitcoin Heist [27]. These represent commercial decision support, medical diagnosis, and security-oriented rare-event detection respectively. In addition, Iris [28] is used as a sanity-check benchmark to verify that the implementations and evaluation pipeline behave sensibly on a small, well-understood multiclass problem.

Taken together, these datasets provide a useful spectrum of tabular classification tasks. Iris acts as a methodological control. Breast Cancer provides a high-stakes diagnostic benchmark. Bank Marketing provides a commercially meaningful imbalanced task. Bitcoin Heist provides a large-scale and security-relevant rare-event detection problem. This combination makes it possible to study not only which algorithms perform best, but also how evaluation choices affect what the results appear to mean.

The project is also directly relevant to my future career interests in artificial intelligence, cybersecurity, and edge-deployed intelligent systems. I am particularly interested in trust-

worthy AI for constrained and safety-critical environments, including automotive security and AI at the edge, where models may need to operate on structured sensor or event data under real deployment constraints. In such settings, it is not enough to apply a library model uncritically. It is necessary to understand how models behave, how evaluation design can mislead, and how performance should be interpreted when false negatives and false positives have different operational consequences. By implementing classical models from scratch and evaluating them under leakage-aware and domain-sensitive protocols, this project has developed knowledge and skills that are directly relevant to future work in secure AI systems, automotive cybersecurity, and reliable machine learning for edge-based decision making.

1.2 Problem Statement

The central problem addressed in this project is how to carry out a trustworthy comparative study of classical supervised learning algorithms on real-world tabular classification tasks in which class imbalance and asymmetric misclassification costs materially affect what should count as good performance.

This problem is both empirical and methodological. Empirically, the project compares k-NN, Decision Trees, and SVMs across datasets that differ in size, feature structure, and domain context. Methodologically, it asks how such a comparison can be made fair and meaningful. A comparison is only useful if the models are implemented correctly, trained under matched conditions, and evaluated using metrics and protocols that reflect the structure of the underlying task. Otherwise, apparent differences between algorithms may simply be artefacts of poor experimental design rather than genuine differences in model suitability [13, 12].

This issue is especially acute in imbalanced classification. On Bank Marketing, a classifier may achieve respectable accuracy while remaining weak at identifying likely subscribers. On Breast Cancer, a high-accuracy model may still be unacceptable if it misses malignant cases. On Bitcoin Heist, naive random splitting can create the illusion of near-perfect performance even when the model fails to generalise to later activity. The problem is therefore not simply to maximise a score, but to determine which algorithms, preprocessing choices, metrics, and evaluation procedures support reliable conclusions in realistic cost-sensitive settings.

Accordingly, this project asks the following broad question: given three algorithmically distinct families of classifier, implemented from scratch within a common software framework, what can be learned about their relative strengths, weaknesses, and practical suitability when they are evaluated rigorously on tabular problems where minority-class performance matters?

1.3 Aims and Objectives

Overall Aim

To implement and empirically compare three foundational supervised learning algorithms, namely k-Nearest Neighbours, Decision Trees, and Support Vector Machines, from scratch within a unified and reproducible experimental framework, and to draw principled conclusions about their behaviour on imbalanced and cost-sensitive tabular classification problems drawn from commercial, medical, and cybersecurity contexts.

Specific Objectives

1. To conduct a critical literature review covering supervised learning theory, inductive bias, the selected algorithm families, evaluation methodology for imbalanced classification, and prior comparative benchmarking studies [1, 2, 3, 19, 17, 18].
2. To design and implement from scratch object-oriented versions of `KNNClassifier`, `DecisionTreeClassifier`, `LinearSVM_Pegasos`, and `OneVsRestSVM`, following a consistent `fit/predict` interface and using the Pegasos stochastic subgradient method for the linear SVM [9].
3. To build a reusable experimental infrastructure supporting data loading, preprocessing, cross-validation, hyperparameter search, threshold tuning where appropriate, metric computation, and machine-readable logging, while ensuring that preprocessing is fitted only on training data within each fold to avoid leakage [13].
4. To validate each from-scratch implementation against the corresponding scikit-learn baseline under matched preprocessing and evaluation conditions [23].
5. To evaluate the implemented models on Bank Marketing, Breast Cancer, and Bitcoin Heist, with Iris used as a sanity-check benchmark.
6. To compare algorithm performance using metrics appropriate to imbalanced classification, including F1-score, Macro-F1, ROC-AUC, and PR-AUC [14, 15, 21].
7. To investigate how preprocessing, hyperparameter tuning, class weighting, and train-test splitting strategy influence the trustworthiness of the conclusions drawn from the experiments.
8. To reflect on the practical implications of the results for real-world decision problems in banking, medicine, and cybersecurity.

1.4 Research Questions

The project is organised around the following research questions.

- RQ1** How closely do from-scratch implementations of k-NN, Decision Trees, and a Pegasos-based linear SVM match the performance of their scikit-learn counterparts under matched preprocessing and evaluation conditions?
- RQ2** How do the three algorithm families differ in classification performance across Bank Marketing, Breast Cancer, and Bitcoin Heist, and are these differences consistent with theoretical expectations arising from their inductive biases?
- RQ3** How does the choice of evaluation metric affect the interpretation of model performance on imbalanced datasets?
- RQ4** How sensitive are experimental conclusions to the evaluation protocol, particularly when random splitting is compared with temporally-aware splitting on datasets with implicit time structure?

1.5 Project Contributions

This project makes five main contributions.

From-scratch implementations of classical classifiers. The project develops Python implementations of three classical classification algorithms: k-Nearest Neighbours, Decision Trees, and a linear Support Vector Machine trained using a Pegasos-style primal subgradient method, together with a one-versus-rest extension for multiclass classification.

A unified experimental framework. The project provides a unified framework for dataset loading, preprocessing, cross-validation, hyperparameter tuning, metric calculation, result logging, and visualisation. This enables more consistent comparison across models and datasets while reducing the risk of data leakage.

Cross-domain empirical analysis of performance in context. By evaluating the same algorithm families across commercial, medical, and cybersecurity datasets, the project shows that model performance must be interpreted in relation to the application context, especially where class imbalance changes the meaning of common evaluation metrics.

Comparison with established library baselines. The project compares the custom implementations against matched scikit-learn baselines, providing a practical reference point for assessing whether the behaviour of the from-scratch models is broadly consistent with established library implementations.

Methodological insight into evaluation design. Finally, the project demonstrates that evaluation design can materially affect benchmark conclusions, most clearly in the Bitcoin Heist case study, where a stricter temporally aware evaluation produced more cautious and credible findings than easier random-split testing..

1.6 Report Structure

Following this introduction, Chapter 2 reviews the theoretical and empirical literature relevant to the project. Chapter 3 describes the design and implementation of the software framework. Chapter 4 presents the experimental methodology, Chapter 5 the results, Chapter 6 the discussion, Chapter 7 the project evaluation and reflection, and the final chapter draws the report to a conclusion.

Chapter 2: Background and Literature Review

This chapter establishes the theoretical and empirical context for the project. It begins with the supervised learning problem and the challenge of generalisation, then examines class imbalance as the main methodological issue shaping the datasets used in this study. It next reviews the three algorithm families implemented in the project before situating the work within the wider comparative literature. The chapter therefore provides the theoretical basis for the design choices and experimental decisions developed later in the report.

2.1 Supervised Learning and Generalisation

In supervised classification, the learner is given a training set

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n, \quad (2.1)$$

where $x_i \in \mathcal{R}^d$ is a feature vector and $y_i \in \mathcal{Y}$ is a class label. The goal is to learn a function f that predicts labels for unseen inputs with low generalisation error. Mitchell [2] formalises learning in terms of experience, task, and performance measure, which is important here because it makes clear that performance depends partly on how success is defined. A model that appears strong under accuracy may appear weaker under recall or PR-AUC.

From a statistical learning perspective, the objective is to minimise expected risk,

$$R(f) = E_{(X,Y)}[L(Y, f(X))]. \quad (2.2)$$

Since the data-generating distribution is unknown, learning proceeds by minimising empirical risk on the observed sample,

$$\hat{R}(f) = \frac{1}{n} \sum_{i=1}^n L(y_i, f(x_i)). \quad (2.3)$$

Low empirical risk does not guarantee low expected risk, so the central problem is not simply fitting the training data but learning a rule that generalises [1].

A useful conceptual framework is the bias-variance trade-off. For a regression estimator $\hat{f}(x)$, the expected prediction error can be decomposed as

$$E[(Y - \hat{f}(x))^2] = \text{Var}(Y | x) + (\text{Bias}[\hat{f}(x)])^2 + \text{Var}[\hat{f}(x)]. \quad (2.4)$$

Although exact for squared error, the same logic carries over conceptually to classification. Increasing k in k-NN, constraining tree depth, or regularising an SVM all alter the balance between local fit and generalisation [1].

Closely related is the notion of *inductive bias*. Mitchell [2] argues that every learner depends on assumptions about the target function beyond the observed data. The algorithms compared in this project embody different biases: k-NN assumes local smoothness, Decision Trees assume recursive axis-aligned partitioning, and SVMs assume that useful separating boundaries can be learned through margin-based optimisation. Comparing them is therefore meaningful not because one should always win, but because their suitability depends on the geometry and difficulty of the dataset.

Generalisation also depends on evaluation design. Hastie et al. [1] distinguish between *model selection*, which chooses hyperparameters, and *model assessment*, which estimates performance on unseen data. If these are not properly separated, performance estimates can

become optimistically biased. This is especially important in comparative work, where small differences between models may otherwise reflect evaluation artefacts rather than genuine superiority.

Two conclusions follow. First, there is no universally best classifier in the abstract. Second, model comparison is only meaningful when the evaluation process is itself trustworthy.

2.2 Class Imbalance in Classification

A classification problem is imbalanced when one class is represented far more frequently than another. If N_{maj} and N_{min} denote the majority and minority counts, a simple summary is

$$IR = \frac{N_{maj}}{N_{min}}. \quad (2.5)$$

However, the literature shows that imbalance is not only a numerical property of the label distribution. Its practical impact depends on class overlap, data geometry, the learning algorithm, and the metric used to evaluate performance [19, 20].

He and Garcia [19] identify imbalanced learning as a major practical challenge because standard classification objectives tend to reward majority-class correctness disproportionately. Japkowicz and Stephen [20] show that the damage caused by imbalance depends strongly on problem complexity. If classes are well separated, even substantial imbalance may be manageable. If minority examples overlap with the majority region, imbalance becomes much more harmful.

This matters directly for the datasets used in this project. Bank Marketing is an imbalanced commercial response problem. Breast Cancer is a high-stakes diagnostic task in which missed malignant cases matter most. Bitcoin Heist is a rarer and more security-oriented detection problem in a noisier behavioural space. In all three, the importance of the task lies not only in class frequency but in the meaning of the errors made on each class.

The literature distinguishes several broad responses to imbalance. Data-level methods alter the training distribution through over- or under-sampling, including methods such as SMOTE [22]. Algorithm-level methods modify the learner through class weights, threshold adjustment, or cost-sensitive objectives [19]. Ensemble approaches attempt to improve minority detection by combining multiple models or multiple resampled views of the data.

This project focuses mainly on algorithm-level responses rather than synthetic resampling. That is a deliberate design choice. The aim is to understand the behaviour of classical algorithms under controlled and comparable conditions, not to maximise performance by layering multiple interventions on top of the learner. This makes the comparison more interpretable and keeps the benchmark centred on the model families themselves.

A direct implication of the imbalance literature is that a model may appear strong overall while still being weak on the cases that matter most. Once minority-class performance becomes important, metric choice becomes part of the problem definition itself.

2.3 Evaluation Metrics for Imbalanced Problems

The evaluation literature shows that different metrics answer different questions, and that this becomes especially important under class imbalance [14, 15, 21]. A classifier that appears strong under one metric may appear weak under another. For this project, evaluation is therefore not a mechanical reporting step but part of how the learning problem is represented.

For binary classification, predictions can be summarised by the confusion matrix, consisting of true positives (TP), false positives (FP), true negatives (TN), and false negatives (FN). Standard metrics follow:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}, \quad (2.6)$$

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad (2.7)$$

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \quad (2.8)$$

$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}. \quad (2.9)$$

In imbalanced problems, accuracy is often the least informative measure because majority-class correctness can dominate the result. Precision, recall, and F_1 provide a more useful summary of minority-class detection. This matters directly for the project datasets: Breast Cancer places special importance on recovering malignant cases, Bank Marketing on balancing missed opportunities against wasted outreach, and Bitcoin Heist on balancing detection against false-alarm burden.

For multiclass problems, averaging strategy also matters. Macro-averaged F_1 is defined as

$$F_1^{\text{macro}} = \frac{1}{K} \sum_{k=1}^K F_{1,k}, \quad (2.10)$$

where K is the number of classes. Macro-averaging gives equal weight to each class and is therefore useful when large classes would otherwise dominate the summary.

Threshold-independent ranking metrics are also important. ROC-AUC summarises discrimination across thresholds by considering the trade-off between true positive rate and false positive rate [14]. However, Davis and Goadrich [15] and Saito and Rehmsmeier [21] show that precision-recall analysis is often more informative in rare-event settings, because precision directly reflects the false-positive burden among predicted positives. This makes PR-AUC especially relevant for Bitcoin Heist.

The metric literature therefore supports a pluralistic approach. No single metric is sufficient. Accuracy is easy to communicate but often misleading under imbalance. F_1 captures the precision-recall trade-off. Macro- F_1 protects minority-class influence in multiclass problems. ROC-AUC summarises ranking quality, while PR-AUC is often more informative in rare-event detection. For this reason, the project adopts a multi-metric evaluation design.

2.4 k-Nearest Neighbours

k-Nearest Neighbours is one of the simplest non-parametric classification methods. Rather than fitting an explicit model during training, it stores the training data and classifies a new

observation by examining the labels of the k closest training points. With Euclidean distance, the distance between x and a training point x_i is

$$d(x, x_i) = \sqrt{\sum_{j=1}^d (x_j - x_{ij})^2}. \quad (2.11)$$

The prediction rule can be written as

$$\hat{y}(x) = \arg \max_{c \in \mathcal{Y}} \sum_{(x_i, y_i) \in \mathcal{N}_k(x)} \mathbf{1}(y_i = c), \quad (2.12)$$

where $\mathcal{N}_k(x)$ is the set of the k nearest neighbours of x .

Cover and Hart [4] showed that the 1-nearest neighbour classifier has a strong asymptotic guarantee: as the sample size tends to infinity, its error rate is bounded above by twice the Bayes error rate. This establishes nearest-neighbour classification as a theoretically respectable learner rather than a mere heuristic. At the same time, the result says little about finite-sample behaviour in noisy, high-dimensional, or poorly scaled spaces, which is where many real tabular problems become difficult.

The central hyperparameter is k . Small k produces a highly local and flexible decision rule, typically lowering bias but increasing variance. Larger k smooths the boundary and stabilises predictions, but may obscure local structure and increase bias [1]. This makes the choice of k a direct expression of the bias-variance trade-off.

k-NN is highly sensitive to feature scaling because distance is computed directly in input space. It is also vulnerable to the curse of dimensionality. Bishop [3] explains that in higher dimensions the distinction between near and far points becomes less informative, making local neighbourhoods less reliable. This suggests, even before experimentation, that k-NN may perform differently on the smaller and more structured Breast Cancer problem than on the more behavioural and difficult Bitcoin Heist setting.

Imbalance introduces an additional weakness. Since prediction is based on local voting, minority examples can be outvoted by nearby majority examples unless the minority class forms sufficiently dense local structure. Japkowicz and Stephen [20] show that local methods can be especially vulnerable when the minority class is sparse and overlaps with the majority region. In this project, k-NN therefore acts not only as a baseline, but also as a test of whether local similarity is a sufficiently strong organising principle for the datasets considered.

2.5 Decision Trees

Decision Trees classify by recursively partitioning the feature space. At each node, the algorithm chooses a feature and split that best separate the classes according to an impurity measure, then repeats this process on the resulting subsets. The final model can be interpreted as a hierarchy of rules, which is one reason trees remain attractive in practical tabular settings.

Quinlan's ID3 algorithm [5] introduced information gain based on entropy. If a node contains class proportions $p_1, \dots, p_K$, its entropy is

$$H(\mathcal{S}) = - \sum_{k=1}^K p_k \log_2 p_k. \quad (2.13)$$

If the sample $\mathcal{S}$ is partitioned into children $\mathcal{S}_1, \dots, \mathcal{S}_m$, the information gain of the split is

$$\text{Gain}(\mathcal{S}) = H(\mathcal{S}) - \sum_{r=1}^m \frac{|\mathcal{S}_r|}{|\mathcal{S}|} H(\mathcal{S}_r). \quad (2.14)$$

The preferred split is the one that produces the greatest reduction in uncertainty.

Breiman et al. [6], in the CART framework, proposed Gini impurity as an alternative criterion:

$$G(\mathcal{S}) = 1 - \sum_{k=1}^K p_k^2. \quad (2.15)$$

The impurity reduction of a candidate split can then be written as

$$\Delta I = I(\mathcal{S}) - \sum_{r=1}^m \frac{|\mathcal{S}_r|}{|\mathcal{S}|} I(\mathcal{S}_r), \quad (2.16)$$

where I is either entropy or Gini impurity.

Decision Trees are appealing because they can represent nonlinear decision boundaries, handle mixed feature types, and produce interpretable decision rules [5, 6]. Unlike distance-based methods, they do not require feature scaling, which makes them well suited to heterogeneous tabular data. Their flexibility also creates instability: fully grown trees can fit noise aggressively and become highly sample-dependent. Breiman et al. [6] showed that controlling tree complexity is therefore essential for good generalisation.

The literature is also useful in a more critical sense. Trees are often praised for interpretability, but interpretability alone does not guarantee balanced performance. Standard impurity criteria do not directly encode asymmetric error costs, so a split that improves overall purity may still be weak in minority-class recovery. Japkowicz and Stephen [20] note that tree-based learners can therefore be sensitive to skew unless they are tuned carefully or extended with cost-sensitive mechanisms. In this project, Decision Trees occupy an informative midpoint between k-NN and SVMs: more structured than k-NN, but still vulnerable to overfitting and class-skew effects if complexity is poorly controlled.

2.6 Support Vector Machines

Support Vector Machines approach classification through regularised optimisation. In the linear case, the classifier predicts according to

$$f(x) = \text{sign}(w^\top x + b), \quad (2.17)$$

where w is the normal vector to the separating hyperplane and b is the intercept. The key idea is to choose the separating boundary with the largest possible margin, since larger margins are associated with stronger generalisation [7, 3].

For non-separable data, Cortes and Vapnik [7] introduced the soft-margin SVM:

$$\min_{w, b, \xi} \quad \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i \quad (2.18)$$

subject to

$$y_i(w^\top x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0. \quad (2.19)$$

The slack variables ξ_i allow margin violations and misclassifications, while C controls the trade-off between fitting the training data and maintaining a wide margin.

An equivalent unconstrained view is obtained through hinge loss,

$$\ell_{\text{hinge}}(x_i, y_i) = \max(0, 1 - y_i(w^\top x_i + b)), \quad (2.20)$$

which leads to the regularised empirical objective

$$\min_{w, b} \quad \frac{\lambda}{2} \|w\|^2 + \frac{1}{n} \sum_{i=1}^n \max(0, 1 - y_i(w^\top x_i + b)). \quad (2.21)$$

This formulation is especially relevant here because it underpins the Pegasos algorithm used in the from-scratch linear SVM implementation.

Shalev-Shwartz et al. [9] propose Pegasos as a primal stochastic subgradient method for optimising this objective. In the single-example case, if (x_i, y_i) violates the margin, the update is

$$w_{t+1} = (1 - \eta_t \lambda) w_t + \eta_t y_i x_i, \quad (2.22)$$

and otherwise

$$w_{t+1} = (1 - \eta_t \lambda) w_t, \quad (2.23)$$

with learning rate

$$\eta_t = \frac{1}{\lambda t}. \quad (2.24)$$

This makes Pegasos attractive for the project because it provides a transparent route to building a linear SVM from first principles.

SVMs are often especially strong in higher-dimensional settings. Bishop [3] and Cristianini and Shawe-Taylor [8] both emphasise that linear SVMs can remain competitive when the feature representation is already informative. This suggests why a linear margin-based model may perform differently on Bank Marketing and Breast Cancer than on Bitcoin Heist. At the same time, SVMs are sensitive to feature scaling, since the margin is defined in feature space. This provides a direct theoretical justification for the standardisation procedures used later in the experimental pipeline.

The literature also suggests a more nuanced relationship between SVMs and imbalance than is often assumed. Because the objective concentrates on support vectors near the decision boundary, SVMs can be more robust than local voting or impurity-based partitioning when minority examples remain informative near that boundary. However, severe skew can still pull the margin toward the minority class unless class weights or threshold adjustments are used. This makes SVMs particularly valuable in this study: they are not guaranteed to win, but there is a clear theoretical reason to expect them to behave differently from k-NN and Decision Trees on more difficult imbalanced tasks.

2.7 Related Work

The comparative literature suggests two broad conclusions. First, no classifier dominates across all tabular datasets. Second, evaluation quality is often as important as algorithm choice itself.

Caruana and Niculescu-Mizil [17] compared multiple supervised learning algorithms across a range of binary classification tasks and showed that performance depended strongly on dataset

structure and on the metric used. Fernández-Delgado et al. [18] extended this perspective to a larger benchmark and again found no universal winner. These studies justify the comparative spirit of this project, but they also limit what it can claim: a responsible benchmark should explain dataset-specific behaviour rather than search for a universally best classifier.

What large benchmark studies usually do not emphasise is the from-scratch implementation perspective. In most cases, classifiers are compared as library artefacts rather than as models implemented, validated, and analysed from first principles. This project therefore adds a software-engineering and implementation dimension that broad benchmark work often leaves implicit.

The domain literature reinforces the importance of this perspective. Moro et al. [25] frame Bank Marketing as a real customer response prediction problem, while Akcora et al. [27] present Bitcoin Heist as a security-oriented detection task. These studies ground the datasets in meaningful applications, but they do not resolve how fair cross-model comparison should be designed.

That methodological counterpart comes from the evaluation literature. Kohavi [10] showed the value of stratified cross-validation. Arlot and Celisse [11] analysed cross-validation procedures more broadly. Cawley and Talbot [12] showed that model selection can itself overfit. Kaufman et al. [13] formalised data leakage and its effect on reported performance. Together, these studies show that even polished benchmarks can be methodologically fragile if evaluation design is weak.

This project therefore sits at the intersection of two strands of literature: empirical comparison of classifier families, and critical analysis of evaluation validity. Its specific contribution is to bring these together in a from-scratch implementation setting with explicit attention to imbalance and evaluation rigour.

2.8 Critical Summary of the Literature

The literature establishes a strong foundation for the project, but also reveals a clear gap.

The theory of supervised learning shows that k-NN, Decision Trees, and SVMs differ in inductive bias rather than merely in coding style or hyperparameter choice [2, 1, 3]. Their behaviour should therefore vary with data geometry, dimensionality, and noise structure. The imbalance literature further shows that model quality cannot be judged by overall accuracy alone. Minority-class detection, metric choice, and asymmetric error costs are central to evaluation in realistic settings [19, 20, 21].

At the same time, the methodological literature shows that trustworthy comparison is itself non-trivial. Cross-validation must be designed carefully, hyperparameter tuning can create optimistic bias, and leakage can enter through preprocessing or feature handling [10, 11, 12, 13]. This means that a technically correct classifier can still produce misleading conclusions if it is embedded in a weak evaluation pipeline.

Broad benchmark studies show that no classifier is universally best [17, 18], while domain papers such as [25] and [27] show that application context changes what should count as useful performance. What remains less well explored is the combination of from-scratch implementation, explicit validation against trusted baselines, multi-metric evaluation under imbalance, and close attention to evaluation validity within a single coherent study.

This project is motivated by exactly that gap. It is not intended to identify a universally

superior classifier. Rather, it seeks to determine how three classical algorithm families behave when they are implemented from first principles, evaluated within a common leakage-resistant framework, and applied to tabular problems in which minority-class performance has genuine practical importance. The next chapter therefore turns from theory to the design of the software system used to carry out that comparison.

Chapter 3: Software Design and Implementation

3.1 Overall System Architecture

The software produced in this project was designed as a compact machine learning experimentation framework rather than as a set of unrelated scripts. The central design objective was to separate model logic, data handling, evaluation, tuning, and result generation so that the same custom implementations could be reused consistently across datasets and compared fairly against scikit-learn baselines.

At a high level, the framework is organised around five components. The first is the `algorithms/` package, which contains the from-scratch implementations of k-Nearest Neighbours, Decision Trees, and Support Vector Machines. The second is the shared evaluation layer in `evaluation/pipeline.py`, which handles cross-validation, metric computation, and fold-local preprocessing. The third is the dataset preparation layer in `data_utils.py`, which converts raw files into experiment-ready matrices. The fourth is the tuning layer in `grid_search.py`. The fifth is the experiment layer, consisting of the run scripts and saved outputs in `results/`.

This separation was important for both software engineering and methodological reasons. It made the codebase easier to debug and extend, but it also ensured that different models could be evaluated under the same preprocessing and validation logic. That consistency is essential in a comparative project, since otherwise apparent performance differences may reflect pipeline differences rather than genuine differences between learning algorithms.

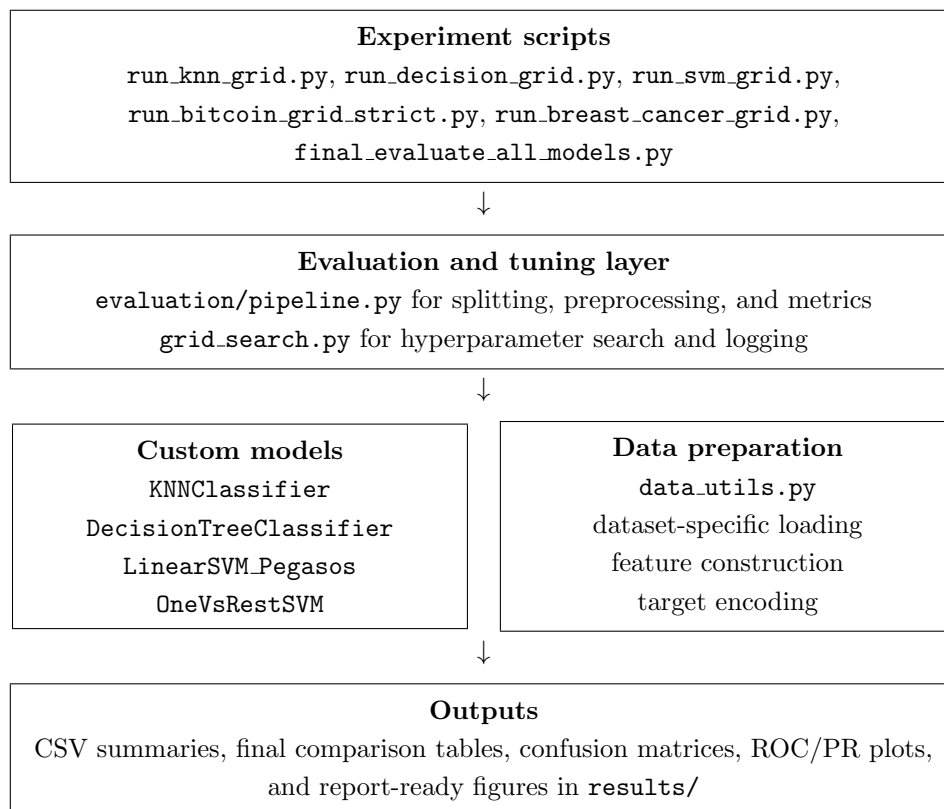


Figure 3.1: High-level architecture of the experimentation framework.

3.2 Repository Structure

The repository is organised around a small number of clearly separated areas, with the main project code, datasets, and generated outputs stored under `src/`. The top-level structure is shown below.

```
PROJECT/
  README.md
  requirements.txt
  Diary.md
  docs/
  results/
  src/
    algorithms/
    evaluation/
    scripts/
      experiments/
      reporting/
      legacy/
    tests/
    data/
      raw/
    results/
    docs/
    data_utils.py
    grid_search.py
    bitcoin_report_utils.py
```

Within this structure, the dependency flow is straightforward. Dataset-specific loading is handled through `data_utils.py`, while shared preprocessing, splitting, cross-validation, and metric computation are concentrated in `evaluation/pipeline.py`. The experiment scripts in `scripts/experiments/` call these shared components to run tuning and final evaluation, and the custom model implementations are contained in `algorithms/`. Outputs such as result tables, prediction files, and figures are written to `src/results/`, while `scripts/reporting/` contains the code used to summarise and visualise those outputs.

This structure was one of the most useful design decisions in the project. It made it possible to refine one part of the methodology, such as fold-local preprocessing or the stricter Bitcoin temporal evaluation pipeline, without rewriting the rest of the codebase. It also helped keep experimental logic, model implementations, reporting utilities, and generated results clearly separated.

3.3 Design Goals and Requirements

Three design goals shaped the implementation.

The first was **algorithmic transparency**. The core training and prediction logic for k-NN, Decision Trees, and the linear SVM was written from scratch using NumPy and standard Python structures. This was necessary because the project is not only about comparing results, but about understanding how the algorithms behave when implemented from first

principles.

The second was **comparability**. Since the report compares custom models with one another and with sklearn baselines, the code had to support a shared evaluation pipeline, consistent preprocessing, and a common `fit/predict` style interface. Without this, the comparison would have been difficult to interpret.

The third was **practical usability**. The framework had to be reproducible, able to save machine-readable outputs, and stable enough to run on datasets larger than the small sanity-check benchmarks. This goal became especially important during debugging, where some early versions were mathematically correct but computationally impractical.

3.4 Algorithm Implementations

3.4.1 k-Nearest Neighbours

The custom k-NN implementation is provided by the class `KNNClassifier` in `algorithms/knn.py`. The main parameters are `n_neighbors`, `weights`, and `p`, allowing the model to vary neighbourhood size, voting rule, and distance metric. The final implementation supports both Euclidean and Manhattan distance.

Prediction is performed by identifying the k nearest training points and aggregating class scores. When `weights = 'uniform'`, each neighbour contributes one vote. When `weights = 'distance'`, the vote weight is inversely proportional to distance. The implementation is vectorised with NumPy, which makes it efficient on smaller and medium-sized datasets.

Algorithm 3.1: k-NN prediction

Input: training data (`X_train`, `y_train`), test point `x`, `k`

1. Compute distances from `x` to all rows in `X_train`
2. Select the indices of the `k` smallest distances
3. Retrieve the labels of those `k` neighbours
4. Aggregate class scores using either:
 - uniform voting, or
 - distance-weighted voting
5. Predict the class with the largest score

A key design issue emerged during larger experiments. The earliest version used fully vectorised all-pairs distance computation, which was correct but could become infeasible in memory on large Bank Marketing and Bitcoin folds. The final implementation therefore introduced batched distance evaluation, preserving the same prediction rule while reducing memory usage substantially. This is an important example of the broader development pattern in the project: correctness had to be complemented by computational practicality.

3.4.2 Decision Tree

The custom tree model is implemented by `DecisionTreeClassifier` in `algorithms/decision_tree.py`, using a recursive binary tree structure built from custom `DecisionNode` objects. The implementation supports both `gini` and `entropy` criteria, together with the main regularisation hyperparameters `max_depth`, `min_samples_split`, `min_samples_leaf`, and `min_impurity_decrease`.

The split search is exhaustive over features and candidate thresholds. At each node, the algorithm chooses the feature-threshold pair that gives the largest impurity reduction. Stopping occurs when the node is pure, when a depth or sample-size constraint is reached, or when no valid split gives sufficient gain.

Algorithm 3.2: Recursive tree growth

Input: sample S at current node

1. If S is pure, return a leaf
2. If stopping criteria are met, return a leaf
3. For each feature:
 - evaluate candidate thresholds
 - compute impurity reduction
4. Select the best valid split
5. Partition S into left and right subsets
6. Recursively grow the left child
7. Recursively grow the right child
8. Return the resulting internal node

This implementation uses pre-pruning only. No post-pruning stage is included. That was a deliberate scope choice: pre-pruning was sufficient to support controlled tuning and fair comparison while keeping the code readable and faithful to the project goals. The model also supports `predict_proba()`, which is useful in later evaluation stages where score-based metrics are required.

3.4.3 Support Vector Machine

The SVM implementation is the most technically involved part of the codebase. The main binary solver is `LinearSVM_Pegasos` in `algorithms/svm_linear.py`, while multiclass classification is handled by `OneVsRestSVM` in `algorithms/svm_multiclass.py`. The solver is strictly linear and internally converts binary labels to $\{-1, +1\}$.

The training procedure follows the Pegasos-style primal subgradient method. This was chosen because it gives a direct connection between the hinge-loss formulation introduced in Chapter 2 and the actual update rule implemented in code, while also being far more realistic to implement from scratch than a full dual SVM solver.

Algorithm 3.3: Pegasos-style linear SVM training

Input: training data (X, y) , C , `max_iter`, `batch_size`

1. Initialise w and b
2. Set $\lambda = 1 / (C * n)$
3. For $t = 1$ to `max_iter`:
 - sample a mini-batch
 - identify margin-violating examples
 - compute subgradient update
 - apply regularisation shrinkage
 - update bias term
 - optionally project / stabilise weights
 - monitor convergence
4. Return final averaged parameters

The final solver includes several features that were added during development: mini-batch updates, early stopping through `tol/patience`, weight averaging, and class-weight support.

These additions were not cosmetic. They were introduced because earlier versions were functional on small binary tasks but not stable enough for larger or more imbalanced experiments. The SVM therefore provides the clearest example in the project of an implementation that improved through iterative testing and correction.

3.5 Data Loading and Preprocessing Components

Data loading and preprocessing are divided between shared logic in `evaluation/pipeline.py` and dataset-specific logic in `data_utils.py`. This separation reflects an important design principle. Generic operations such as imputation and scaling are shared. Dataset-specific transformations such as one-hot encoding for Bank Marketing or dropping the address column in Bitcoin Heist are handled where the dataset semantics are known.

The custom `DataLoader` supports both CSV and ARFF files. Missing values are mapped to `np.nan` at load time using a set of common markers. Generic preprocessing is then handled through the `DataPreprocessor` class, which supports missing-value handling, target encoding, and z-score scaling.

The most important methodological feature is that preprocessing is fitted inside each fold. In `cross_validate()`, a fresh `DataPreprocessor` is created for each training fold, fitted only on that fold, and then applied to the corresponding validation fold. This prevents leakage through scaling or imputation statistics.

3.6 Evaluation Pipeline

The shared evaluation logic is implemented in `evaluation/pipeline.py`. This file provides the custom `cross_validate()` function, the splitters used in different experiments, and the main metric computation routines. Centralising this logic was one of the strongest design decisions in the project, because it kept evaluation behaviour consistent across datasets and models.

The standard path uses stratified cross-validation so that class proportions are preserved as closely as possible across folds. The same pipeline is reused for custom models and, where needed, for sklearn baselines. This avoids a situation in which one family of models is evaluated under a different preprocessing or scoring protocol from another.

The evaluation layer also supports stricter grouped validation. This was especially important in the Bitcoin Heist experiments, where grouped or temporally-aware evaluation was needed to avoid overly optimistic random splits.

3.7 Hyperparameter Search Framework

Hyperparameter tuning is implemented in `grid_search.py`. The search routine evaluates each parameter combination by calling the shared cross-validation pipeline and recording the resulting summary metrics in CSV form. The primary selection criterion is `mean_f1`, which is appropriate for a project focused on imbalanced classification rather than raw accuracy.

The search spaces differ by model. For k-NN, the main tuned parameters are neighbourhood size, voting type, and distance metric. For Decision Trees, they are split criterion and pre-pruning controls. For the SVM, they are regularisation strength, iteration count, batch size, and class weighting. The best-performing parameter combinations are then summarised in `best_hyperparams_summary.csv` for later use in final model assessment.

Threshold tuning is kept conceptually separate from model tuning, even though the easier Bitcoin path allows threshold evaluation during grid search. This is methodologically preferable because it distinguishes the process of fitting a model from the later decision of where to set an operating threshold.

3.8 Visualisation and Results Generation

The framework stores outputs primarily as CSV files and then uses separate plotting scripts to generate report-ready figures. This design keeps the core training code separate from presentation logic and makes it possible to regenerate plots without rerunning full experiments.

The main outputs include grid-search summaries, final comparison tables, confusion matrices, ROC and PR curves, and comparison plots. These are saved in `results/` and form the evidence base for the later results chapter. A particularly important artefact is the Bitcoin PR curve output, since it directly supports the rare-event evaluation of the security case study.

3.9 Testing and Validation

Testing and validation were iterative rather than being treated as a single final check. Several parts of the final system were revised after earlier working versions exposed practical or methodological weaknesses.

For k-NN, the key issue was scalability. The earliest implementation was correct on smaller tasks but became impractical on larger folds because of memory-intensive distance computation. Batched distance evaluation was introduced to solve this.

For Decision Trees, the core structure was comparatively stable, but the evaluation around it improved over time. Early experiments relied more heavily on accuracy, whereas later revisions made precision, recall, and F_1 central to the assessment of tree behaviour.

For the SVM, the development path was more substantial. Early versions were functional but comparatively basic. Later versions incorporated a more faithful Pegasos formulation, stronger optimisation control, weight averaging, class weighting, and multiclass support. This makes the SVM the clearest example of an implementation that matured through repeated testing rather than emerging fully formed.

The most important methodological debugging story concerned Bitcoin Heist. Early random-split experiments produced extremely strong results. Rather than accepting these scores uncritically, the later design treated them as potentially optimistic and introduced grouped validation, temporal holdout, and removal of time-related fields. This change materially improved the credibility of the final methodology.

3.10 Reproducibility and Documentation

Reproducibility was a consistent design goal. The repository contains a `README.md`, reusable experiment scripts, saved CSV outputs, and a top-level pipeline runner. Fixed random seeds are used in many parts of the framework, which improves repeatability across tuning and evaluation.

At the same time, reproducibility is stronger at the script-and-output level than at the environment-management level. The repository supports rerunning experiments and regenerating outputs, but dependency pinning is weaker than would be expected in a fully portable research package. This should be recognised as a limitation, although it does not undermine the core experimental logic of the project.

Taken together, the software described in this chapter is best understood as a research-oriented experimentation framework: modular enough to support rigorous comparison, transparent enough to expose algorithmic behaviour, and stable enough to generate reproducible evidence for the later results chapters.

Chapter 4: Experimental Methodology

This chapter explains how the empirical study was conducted. The central methodological objective was to ensure that comparisons between models were fair, leakage-aware, and interpretable in the context of imbalanced classification. The chapter therefore moves from the datasets themselves, to preprocessing and validation design, to the tuning and baseline comparison procedures that generated the final outputs.

4.1 Datasets

Four datasets were used in the final experimental framework: Iris, Breast Cancer Wisconsin Diagnostic, Bank Marketing, and Bitcoin Heist. These datasets were selected to provide a progression from a small and well-understood methodological control to larger and more consequential real-world decision problems.

Dataset	Instances	Features	Classes	Role in the project
Iris	150	4	3	Sanity-check benchmark for correctness and multiclass handling
Breast Cancer Wisconsin Diagnostic	569	30 substantive diagnostic features ¹	2	High-stakes medical classification benchmark
Bank Marketing (bank-full)	45,211	16 raw features before encoding	2	Main commercial imbalanced classification case study
Bitcoin Heist	very large blockchain-derived table ¹	numerical behavioural features	2 in this project	Main security-focused rare-event detection case study

Table 4.1: Core datasets used in the final experimental framework.

4.1.1 Iris Dataset

The Iris dataset was loaded from `data/raw/iris/iris.data`. It contains 150 examples, four numerical features, and three balanced classes of 50 examples each. Within this project, Iris functions mainly as a methodological control. It is useful because it is small, well-understood, and suitable for checking that the custom models, the shared pipeline, and the multiclass extension of the SVM behave sensibly before more demanding experiments are considered.

¹The raw Breast Cancer file also contains an identifier column and a diagnosis column. The clinically meaningful predictive variables are the 30 real-valued diagnostic features.

⁰The repository uses a binary version of Bitcoin Heist in which `white` is mapped to class 0 and all ransomware-related labels are mapped to class 1.

4.1.2 Breast Cancer Dataset

The Breast Cancer Wisconsin Diagnostic dataset was loaded from `data/raw/breast+cancer+wisconsin+di`. It contains 569 examples and a binary diagnosis label, with 357 benign and 212 malignant cases. This corresponds to an approximate class split of 62.7% benign and 37.3% malignant. The clinically meaningful predictive attributes are the 30 real-valued measurements derived from digitised fine needle aspirate images. The raw file also contains an identifier field, which has no diagnostic meaning and should not be treated as a substantive predictive variable.

This dataset is one of the main final-comparison tasks in the report. It is especially valuable because the consequences of false negatives are clear: a classifier that appears strong globally may still be poor if it fails to recover malignant cases reliably.

4.1.3 Bank Marketing Dataset

The main banking dataset used in the project is `bank-full.csv`, loaded from `data/raw/bank+marketing/ba`. The target variable is `y`, mapped from `no/yes` to 0/1. The dataset contains 45,211 examples. The positive class, term deposit subscription, is relatively rare, representing roughly 11.7% of the full dataset [25]. This makes Bank Marketing a realistic imbalanced commercial classification task rather than a balanced toy benchmark.

The dataset is central to the report because it provides a commercially meaningful decision problem in which false positives waste outreach resources while false negatives miss potentially valuable customers. It also contains features whose real-world interpretability requires care. In particular, variables such as call duration are highly predictive but must be interpreted cautiously when discussing deployability, because they may not be available in the same form at the moment a real decision is required [25].

4.1.4 Bitcoin Heist Dataset

The Bitcoin Heist dataset was loaded from `data/raw/Bitcoin/BitcoinHeistData.csv` through the custom Bitcoin loader. In this project, it is used in binary form, where `white` is mapped to class 0 and all ransomware-related labels are mapped to class 1. The address field is removed, leaving a set of numerical behavioural features derived from blockchain transaction patterns.

Bitcoin Heist is one of the most important datasets in the report because it combines class imbalance, security relevance, and implicit temporal structure. It therefore serves two roles at once. It is an application dataset for ransomware-related activity detection, but it is also a case study in trustworthy evaluation design. Early random-split experiments produced unusually strong results, which prompted the later move toward grouped and temporally-aware evaluation. For that reason, Bitcoin Heist is central not only to the results chapter, but also to the methodological argument of this report.

4.2 Preprocessing Strategy

Preprocessing was designed to balance consistency across datasets with dataset-specific handling where the raw structure required it. Shared operations are implemented in `evaluation/pipeline.py`, while dataset-specific transformations are implemented in `data_utils.py`.

At load time, the custom **DataLoader** reads CSV or ARFF files and maps common missing-value markers to `np.nan`. Generic preprocessing is then handled through the **DataPreprocessor** class, which supports missing-value handling, label encoding for non-numeric targets, and z-score standardisation for numerical features.

The most important leakage-control rule is that imputation and scaling are fitted inside each fold. During cross-validation, a fresh preprocessor is created for every training fold, fitted only on that fold, and then applied to the corresponding validation fold. This ensures that summary statistics from held-out data do not leak into model fitting.

Dataset	Preprocessing applied
Iris	Numerical features retained; label encoding where needed; fold-local imputation and scaling when required by the model.
Breast Cancer	Numerical features retained; diagnosis used as target; fold-local imputation and scaling; identifier field treated as non-substantive meta-data and should be excluded from the final predictive feature set.
Bank Marketing	<code>y</code> mapped to 0/1; feature matrix one-hot encoded with <code>pd.get_dummies()</code> ; fold-local imputation and scaling for distance- and margin-based models.
Bitcoin Heist	Address column removed; target binarised; numerical feature matrix retained; in the stricter path, <code>year</code> and <code>day</code> are removed before evaluation.

Table 4.2: Dataset-specific preprocessing steps used in the experimental pipeline.

This design is methodologically appropriate for the project because it centralises the most leakage-sensitive operations while still allowing datasets to be represented in a way that respects their actual structure.

4.3 Experimental Protocol

The experimental protocol was designed to separate model development from final model assessment wherever possible. In practical terms, this means that cross-validation was used for hyperparameter search, while a later held-out evaluation was used for the final custom-versus-sklearn comparison.

For the standard tuning path, the repository uses 5-fold stratified cross-validation. This choice provides a practical balance between stable performance estimation and computational tractability, while also preserving class proportions across folds. Within each fold, the same sequence is followed: split the data, fit preprocessing on the training fold only, train the model, and compute metrics on the validation fold.

For the final comparison stage, the project uses a fixed held-out split constructed through the repository’s split utility with a fixed random seed. The exact split is shared between the custom and sklearn models, and the same preprocessing path is applied to both. This makes the final comparison much more interpretable than a comparison based on different splits or different preprocessing conventions.

Bitcoin Heist requires a stricter protocol than the other datasets. The repository contains both an easier random-split path and a stricter path. The easier path was useful during development, but the more credible evaluation is the strict path, which uses grouped validation by year and a temporal holdout in which training data comes from years before 2017 and

the test set is drawn from 2017. In that path, the time-related fields `year` and `day` are also removed. This is the protocol on which the strongest final claims about Bitcoin performance should rest.

4.4 Hyperparameter Tuning Strategy

Hyperparameter tuning is performed through the custom search framework in `grid_search.py`. The search routine evaluates each parameter combination by calling the shared cross-validation pipeline and storing the resulting summary metrics in CSV format. This keeps tuning behaviour consistent across models and avoids the need to wrap the custom models in external tuning code.

Model selection is based primarily on `mean_f1`. This is a deliberate methodological choice. Since the report focuses on imbalanced classification and meaningful recovery of minority or high-consequence cases, accuracy alone would have been an insufficient tuning objective.

The main tuned hyperparameters were as follows. For k-NN, neighbourhood size, voting strategy, and distance metric were varied. For Decision Trees, the split criterion and the main pre-pruning controls were varied. For the SVM, regularisation strength, iteration budget, batch size, and class weighting were varied. The best rows from the grid-search outputs were then summarised in `best_hyperparams_summary.csv` for use in later final evaluation.

Threshold tuning is treated as a separate methodological step. In the Bitcoin experiments, threshold analysis was explored explicitly because a classifier with good ranking performance can still behave very differently depending on where the decision boundary is placed in score space.

4.5 Baseline Comparison with scikit-learn

A central methodological feature of the project is the comparison between the custom implementations and trusted sklearn baselines. This is not used merely as a convenience check. It is the main external reference for judging whether the from-scratch models are behaving plausibly.

The baselines used are `KNeighborsClassifier`, `DecisionTreeClassifier`, and `LinearSVC`. The main final comparison is implemented in `final_evaluate_all_models.py`, which constructs custom and sklearn models for the same dataset, trains them on the same processed arrays, and evaluates them on the same held-out split.

This design is important because it removes many obvious confounds. If custom and library models were trained on different splits or under different preprocessing conditions, any observed gap could not be interpreted confidently. Here, the purpose of the comparison is not to “beat” sklearn in a general sense, but to show whether the custom implementations achieve near-parity under controlled conditions.

4.6 Performance Metrics

The conceptual meaning of the evaluation metrics was discussed in Chapter 2. In this chapter, the emphasis is therefore on which metrics were reported for which datasets and why.

Dataset	Primary reported metrics	Rationale
Iris	Accuracy, Macro- F_1	Balanced multiclass sanity-check benchmark; useful for correctness and multiclass behaviour.
Breast Cancer	Accuracy, Precision, Recall, F_1 , ROC-AUC where available	High-stakes binary task in which recovery of malignant cases matters.
Bank Marketing	Accuracy, Precision, Recall, F_1 , ROC-AUC, Average Precision where available	Imbalanced commercial task in which both missed opportunities and wasted outreach matter.
Bitcoin Heist	Precision, Recall, F_1 , ROC-AUC, Average Precision / PR-based analysis	Rare-event security task in which ranking quality and false-positive burden are central.

Table 4.3: Metric emphasis across datasets in the final evaluation design.

The use of F_1 as the main tuning criterion is consistent with this design. The inclusion of ROC-AUC and average precision where score outputs are available is especially important for Bitcoin Heist, where threshold-independent assessment is more informative than a single hard-decision score.

4.7 Threats to Validity

Although the final methodology is strong overall, several limitations should be stated clearly.

The most important threat concerns Bitcoin Heist. The existence of both an easier random-split path and a stricter temporal path shows that the evaluation protocol can materially affect the apparent strength of the models. For that reason, the stricter temporal design is the more credible basis for final interpretation.

A second threat concerns raw feature handling in the Breast Cancer data. The raw file contains an identifier column in addition to the diagnosis label and the substantive diagnostic measurements. Since the identifier has no clinical meaning, it should not contribute to the predictive feature set. Any experiment retaining it would reduce methodological realism.

A third threat concerns consistency between tuning and final evaluation. Decision Trees are often tuned without scaling in their dedicated scripts, which is conceptually reasonable, but the final comparison script scales all models for consistency. This does not invalidate the final comparison, but it does mean that tuning and assessment conditions are not perfectly identical for every model family.

A fourth limitation is asymmetry in tuning coverage. In particular, Bitcoin k-NN is weaker methodologically than the other main model-dataset combinations because computational constraints limited its full tuning path.

A fifth limitation is that the project reports strong descriptive comparisons rather than formal paired significance tests across all models and datasets. The comparative conclusions should therefore be read as methodologically careful descriptive findings, not as universal inferential claims.

Despite these limitations, the methodology remains strong overall. The shared evaluation core, fold-local preprocessing, explicit baseline comparison, saved outputs, and stricter Bitcoin redesign together provide a credible basis for the results presented in the next chapter.

Chapter 5: Results

This chapter presents the empirical results produced by the experimental framework described in Chapters 3 and 4. Its purpose is primarily descriptive, but it also includes paired prediction-level comparison between the custom and scikit-learn implementations on the same final test sets. This strengthens the evaluation beyond aggregate metrics alone and makes it possible to distinguish between near-parity in final hard predictions and more substantive divergence in operating behaviour.

The authoritative evidence base for this chapter comes from two final comparison paths. For Iris, Bank Marketing, and Breast Cancer, the final comparison is taken from the shared all-dataset evaluation pipeline. For Bitcoin Heist, the headline result is taken from the newer report-era strict pipeline, which supersedes the earlier optimistic random-split Bitcoin outputs. This distinction is important because one of the strongest findings in the project is that evaluation design materially changed the apparent Bitcoin conclusion.

5.1 Implementation Validation Against scikit-learn

A central aim of the project was to determine whether the from-scratch implementations behaved plausibly when compared with trusted reference models under matched preprocessing and evaluation conditions. Figure 5.1 summarises the final non-Bitcoin comparison, while Table 5.1 gives the corresponding final F1-scores.

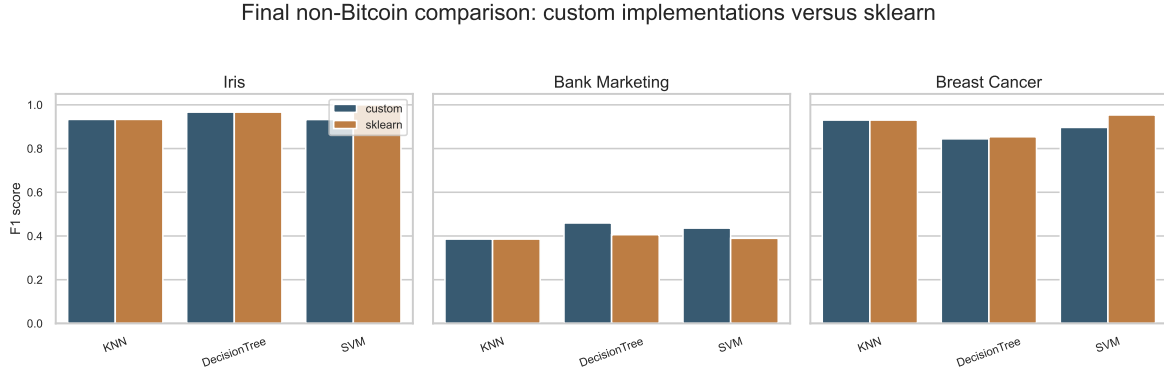


Figure 5.1: Final non-Bitcoin comparison between the custom implementations and the corresponding scikit-learn baselines.

Dataset	KNN F1		Decision Tree F1		SVM F1	
	Custom	sklearn	Custom	sklearn	Custom	sklearn
Iris	0.9333	0.9333	0.9666	0.9666	0.9327	1.0000
Bank Marketing	0.3858	0.3858	0.4594	0.4059	0.4360	0.3894
Breast Cancer	0.9302	0.9302	0.8444	0.8539	0.8966	0.9535
Bitcoin Heist	—	—	0.0393	0.0392	0.0204	0.0080

Table 5.1: Final F1-scores for the custom implementations and the corresponding scikit-learn baselines. Bitcoin KNN is omitted because it was not part of the final strict Bitcoin report pipeline.

The broad descriptive pattern is that implementation parity is strongest on the standard benchmarks. On Iris, custom KNN and custom Decision Tree match scikit-learn exactly on the final holdout. On Bank Marketing and Breast Cancer, parity remains broadly reasonable

but not perfect. On Bank Marketing, the custom Decision Tree and custom SVM both exceed the corresponding scikit-learn baselines in final F1, whereas on Breast Cancer the scikit-learn SVM remains clearly stronger than the custom SVM. Bitcoin behaves differently from the standard benchmarks. Under the strict report-era protocol, the custom and scikit-learn Decision Trees are almost identical on F1, but the SVM pair diverges sharply in operating behaviour.

Taken together, these results show that the custom implementations are sufficiently faithful to support meaningful comparison, especially for KNN and Decision Trees, while also showing that implementation parity is most stable on conventional tabular benchmarks and becomes more sensitive on the more difficult Bitcoin task.

5.2 Standard Benchmark Results

5.2.1 Iris

The Iris dataset was used primarily as a sanity-check benchmark. It is a balanced three-class task and therefore provides a useful control case for verifying that the pipeline and the multiclass wrapper behave sensibly.

Model Family	Implementation	Accuracy	Precision	F1
KNN	Custom	0.9333	0.9333	0.9333
KNN	sklearn	0.9333	0.9333	0.9333
Decision Tree	Custom	0.9667	0.9697	0.9666
Decision Tree	sklearn	0.9667	0.9697	0.9666
SVM	Custom	0.9333	0.9444	0.9327
SVM	sklearn	1.0000	1.0000	1.0000

Table 5.2: Final Iris holdout results.

The Iris results are strong overall. Both KNN implementations produce the same final scores, and the custom and sklearn Decision Trees also match exactly. The main remaining gap occurs in the SVM comparison, where the custom multiclass SVM reaches an F1-score of 0.9327 while the sklearn SVM achieves perfect classification on this holdout. Even so, the custom SVM remains competitive rather than failing outright, which is sufficiently strong for a from-scratch undergraduate implementation.

5.2.2 Breast Cancer

The Breast Cancer dataset provides the clearest strong-performing binary benchmark in the report. Since malignant-case recovery matters more than headline accuracy alone, F1, ROC-AUC, and average precision are especially informative here.

The strongest final Breast Cancer model is the sklearn SVM, which achieves 0.9535 F1, 0.9945 ROC-AUC, and 0.9925 average precision. The custom SVM is the strongest of the from-scratch models on this dataset, reaching 0.8966 F1 together with 0.9903 ROC-AUC and 0.9856 average precision. KNN is also strong, with both the custom and sklearn versions reaching identical final F1-scores of 0.9302. By contrast, the Decision Tree family is clearly weaker here than the margin-based or instance-based alternatives.

Model Family	Implementation	Accuracy	Precision	Recall	F1	AP / ROC-AUC
KNN	Custom	0.9478	0.9302	0.9302	0.9302	–
KNN	sklearn	0.9478	0.9302	0.9302	0.9302	0.9499 / 0.9740
Decision Tree	Custom	0.8783	0.8085	0.8837	0.8444	–
Decision Tree	sklearn	0.8870	0.8261	0.8837	0.8539	0.8392 / 0.9088
SVM	Custom	0.9217	0.8864	0.9070	0.8966	0.9856 / 0.9903
SVM	sklearn	0.9652	0.9535	0.9535	0.9535	0.9925 / 0.9945

Table 5.3: Final Breast Cancer holdout results. AP denotes average precision.

A caveat should also be stated explicitly. In the authoritative Breast Cancer pipeline, the identifier column was retained, meaning that the final authoritative runs use 31 features rather than a purely identifier-free 30-feature representation. That does not invalidate the results, but it should be kept in mind when interpreting them later.

5.2.3 Bank Marketing

Bank Marketing is a more difficult and more obviously imbalanced task than Iris. Here, precision, recall, and F1 are more informative than accuracy alone because the positive class represents only a minority of cases in the dataset.

Model Family	Implementation	Accuracy	Precision	Recall	F1	AP / ROC-AUC
KNN	Custom	0.8846	0.5337	0.3021	0.3858	–
KNN	sklearn	0.8846	0.5337	0.3021	0.3858	0.3172 / 0.7321
Decision Tree	Custom	0.8862	0.5346	0.4028	0.4594	–
Decision Tree	sklearn	0.8817	0.5105	0.3368	0.4059	0.4940 / 0.8702
SVM	Custom	0.8954	0.6178	0.3368	0.4360	0.5509 / 0.8900
SVM	sklearn	0.8942	0.6328	0.2812	0.3894	0.5608 / 0.8961

Table 5.4: Final Bank Marketing holdout results. AP denotes average precision.

The strongest custom model on Bank Marketing is the Decision Tree, which achieves an F1-score of 0.4594. This is the highest Bank Marketing F1-score in the final authoritative comparison and exceeds the sklearn Decision Tree baseline, which reaches 0.4059. The custom SVM is also competitive, achieving 0.4360 F1, while the KNN pair is weaker and identical at 0.3858 F1.

A second descriptive point is that the custom models do not behave erratically relative to sklearn. The custom Decision Tree improves recall over the sklearn tree while maintaining similar precision, and the custom SVM improves recall over the sklearn SVM at a modest cost in precision. The accuracy values are all high, ranging from roughly 0.88 to 0.90, but the ranking by F1 is more informative because it exposes more meaningful differences in minority-class detection.

As with Breast Cancer, a caveat should be stated. The authoritative Bank Marketing runs retain the feature **duration**, which can be highly predictive but may not be available in the same way at real decision time. The results reported here are therefore authoritative for the current pipeline, but they should be interpreted with that dataset design choice in mind.

5.3 Bitcoin Heist Under the Strict Report Pipeline

Bitcoin Heist is the most methodologically sensitive dataset in the project. For this reason, the headline Bitcoin result is not taken from the older random-split outputs, but from the newer strict report pipeline that uses grouped cross-validation over pre-holdout years and a final evaluation on an unseen 2017 holdout.

5.3.1 Strict tuning results

The top configurations selected by the strict Bitcoin tuning stage are summarised in Table 5.5. These are the configurations used to support the final holdout evaluation.

Model	Selected configuration	Class weight	Best threshold	CV mean F1	CV AP	CV ROC-AUC
Decision Tree	gini; max_depth = 8; min_samples_split = 20; min_samples_leaf = 20	balanced	0.5811	0.0432	0.0216	0.5606
SVM	C = 0.2; max_iter = 100; batch_size = 256	balanced	0.7653	0.0284	0.0154	0.5079

Table 5.5: Best strict Bitcoin configurations selected during grouped cross-validation on pre-2017 data.

These tuning results are already much weaker than the earlier optimistic Bitcoin outputs. The best cross-validated Decision Tree configuration reaches a mean F1-score of 0.0432, while the best SVM configuration reaches 0.0284. Even before the final holdout is considered, the strict pipeline therefore presents Bitcoin as a difficult and low-signal task rather than as a near-solved problem.

5.3.2 Strict 2017 holdout results

Model Family	Implementation	Accuracy	Precision	Recall	F1	AP	ROC-AUC
Decision Tree	Custom	0.8509	0.0209	0.3221	0.0393	0.0189	0.6709
Decision Tree	sklearn	0.8515	0.0209	0.3204	0.0392	0.0194	0.6747
SVM	Custom	0.1126	0.0103	0.9771	0.0204	0.0079	0.4338
SVM	sklearn	0.9804	0.0077	0.0083	0.0080	0.0082	0.4560

Table 5.6: Strict Bitcoin holdout results on the unseen 2017 evaluation set. AP denotes average precision.

Several descriptive findings stand out. First, the strongest strict Bitcoin model in the final report pipeline is the custom Decision Tree, with $F1 = 0.0393$, $\text{recall} = 0.3221$, and average precision = 0.0189. This is only marginally different from the sklearn Decision Tree, which achieves $F1 = 0.0392$. Second, the SVM family behaves very differently. The custom SVM is extremely recall-oriented, recovering 97.7% of the positive class at the chosen threshold, but with very low precision and very low accuracy. By contrast, the sklearn SVM remains highly accurate overall but recovers almost none of the positive cases, resulting in an F1-score of only 0.0080.

These values make clear why accuracy is a poor headline metric for Bitcoin Heist. The sklearn SVM reaches 0.9804 accuracy while still recovering less than 1% of the positive cases. Conversely, the custom SVM reaches 0.9771 recall while collapsing in accuracy. On this dataset, precision, recall, F1, and average precision provide a much more faithful summary of model behaviour than accuracy alone.

5.4 Paired Prediction-Level Comparison

Aggregate metrics alone do not show whether the custom and sklearn implementations behave similarly on the same final test cases. To address this, paired prediction exports were generated from the authoritative final pipelines and compared directly on the shared final holdout rows. This makes it possible to distinguish implementation near-parity from genuine divergence in operating behaviour.

Dataset	Model	n	Agreement rate	Custom only correct	sklearn only correct	Custom acc.	sklearn acc.
iris	KNN	30	1.0000	0	0	0.9333	0.9333
iris	Decision Tree	30	1.0000	0	0	0.9667	0.9667
iris	SVM	30	0.9333	0	2	0.9333	1.0000
bank	KNN	9043	1.0000	0	0	0.8900	0.8900
bank	Decision Tree	9043	0.9767	109	102	0.8975	0.8967
bank	SVM	9043	0.9492	160	299	0.8838	0.8991
breast_cancer	KNN	115	1.0000	0	0	0.9478	0.9478
breast_cancer	Decision Tree	115	0.9913	0	1	0.8783	0.8870
breast_cancer	SVM	115	0.9565	2	3	0.9565	0.9652
bitcoin	Decision Tree	368486	0.9935	1088	1312	0.8509	0.8515
bitcoin	SVM	368486	0.1138	3377	323163	0.1126	0.9804

Table 5.7: Paired custom-versus-sklearn comparison on the shared final test sets. “Custom only correct” counts cases where the custom model is correct and sklearn is wrong, while “sklearn only correct” counts the reverse.

Dataset	Model	b	c	p -value	Sig. at 0.05	Test type
iris	KNN	0	0	–	No	Not informative
iris	Decision Tree	0	0	–	No	Not informative
iris	SVM	0	2	0.5000	No	Exact
bank	KNN	0	0	–	No	Not informative
bank	Decision Tree	109	102	0.6796	No	Approximate
bank	SVM	160	299	1.18×10^{-10}	Yes	Approximate
breast_cancer	KNN	0	0	–	No	Not informative
breast_cancer	Decision Tree	0	1	1.0000	No	Exact
breast_cancer	SVM	2	3	1.0000	No	Exact
bitcoin	Decision Tree	1088	1312	5.31×10^{-6}	Yes	Approximate
bitcoin	SVM	3377	323163	$< 10^{-16}$	Yes	Approximate

Table 5.8: McNemar-style paired comparison of custom and sklearn hard predictions on the shared final test sets. b denotes cases where the custom model is correct and sklearn is wrong; c denotes the reverse.

Rows marked “Not informative” had no discordant pairs. Rows marked “Exact” used the exact McNemar procedure because discordant counts were very small, while rows marked “Approximate” used the continuity-corrected approximation because the discordant counts were larger.

The paired results strengthen the implementation-correctness story on the standard datasets. Exact agreement is observed for both Iris KNN and Iris Decision Tree, and also for Bank KNN and Breast Cancer KNN. More generally, most standard-dataset custom/sklearn pairs show high agreement rates and only small differences in case-level accuracy. By contrast, the largest raw disagreement count occurs on strict Bitcoin SVM, where the custom and sklearn implementations disagree on 326,540 holdout cases. This is far larger than any other pair and indicates a substantive divergence in operating behaviour rather than a minor numerical discrepancy.

Only three dataset-model pairs show statistically significant paired differences at the 0.05 level: Bank SVM, Bitcoin Decision Tree, and Bitcoin SVM. These results should be inter-

preted cautiously. They reflect paired disagreement in hard predictions on the shared final test sets, not universal superiority of one implementation across all metrics. This distinction is especially important for Bitcoin, where the main issue is not simply aggregate performance level but the radically different decision behaviour induced by thresholded scores.

5.5 Threshold Behaviour on Strict Bitcoin

The paired results show that the largest implementation divergence occurs on strict Bitcoin SVM. To understand that divergence more clearly, it is useful to examine how precision, recall, and F1 change as the decision threshold varies under the strict holdout protocol.

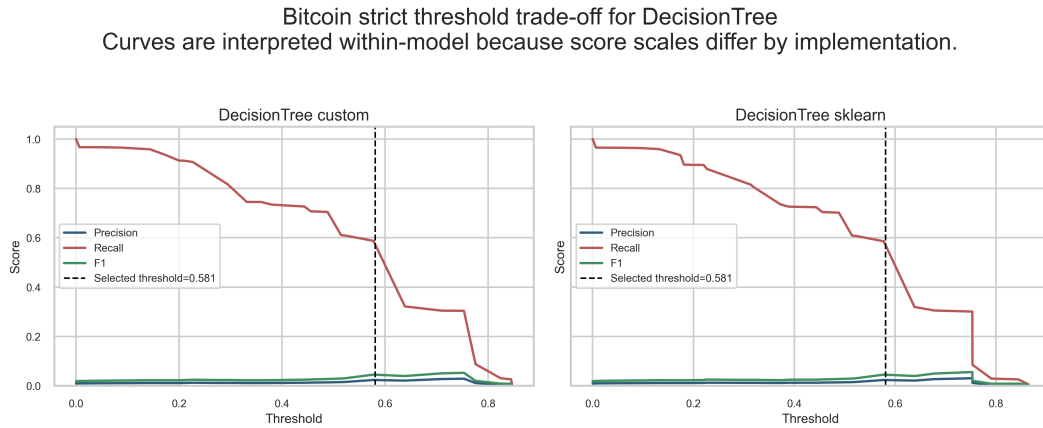


Figure 5.2: Threshold trade-off for the strict Bitcoin custom Decision Tree. The selected threshold is marked explicitly in the figure.

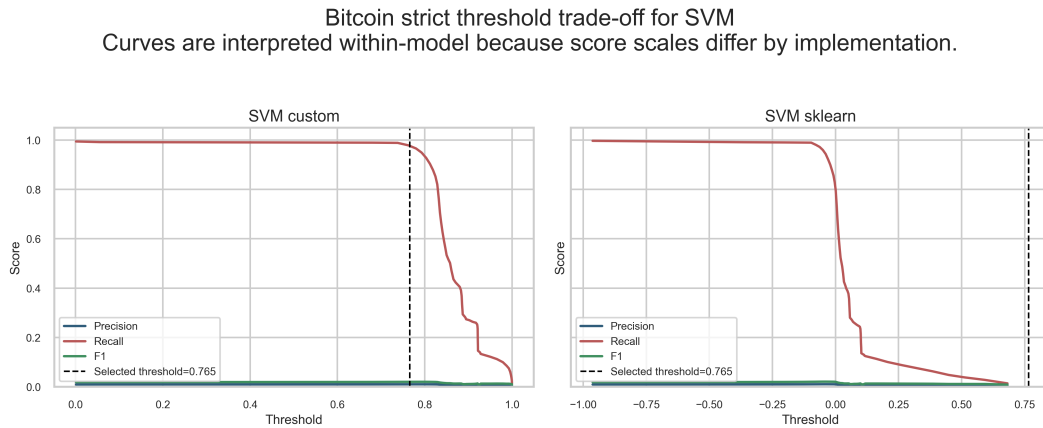


Figure 5.3: Threshold trade-off for the strict Bitcoin custom SVM. The selected threshold is marked explicitly in the figure. Score magnitudes should be interpreted within-model rather than compared numerically across model families.

These threshold-tradeoff plots clarify the difference between the two model families. For the Decision Tree, the selected threshold operates over probability-like leaf scores and leads to a modest but interpretable trade-off between false positives and minority-class recovery. For the SVM, by contrast, threshold choice does not rescue the model into a strong detector. Instead, the selected threshold places the custom SVM in a very high-recall, very low-precision operating regime, while the sklearn SVM remains extremely conservative.

The paired-analysis outputs make this especially clear. Under the selected strict threshold, the custom SVM predicts the positive class on 89.64% of holdout cases, whereas the sklearn

SVM predicts the positive class on only 1.03% of cases. This shows that the SVM divergence is driven primarily by different thresholded operating behaviour on the same strict Bitcoin holdout. At the same time, score values should be interpreted within-model rather than compared numerically across different model families, because some score columns represent probability-like outputs while others represent decision-function margins.

5.6 Effect of Methodological Corrections on Bitcoin Heist

One of the most important findings in the entire project is that the Bitcoin conclusion changed dramatically once the evaluation protocol was tightened. Figure 5.4 shows the contrast between the older easy random-split Bitcoin results and the newer strict temporal-holdout results.

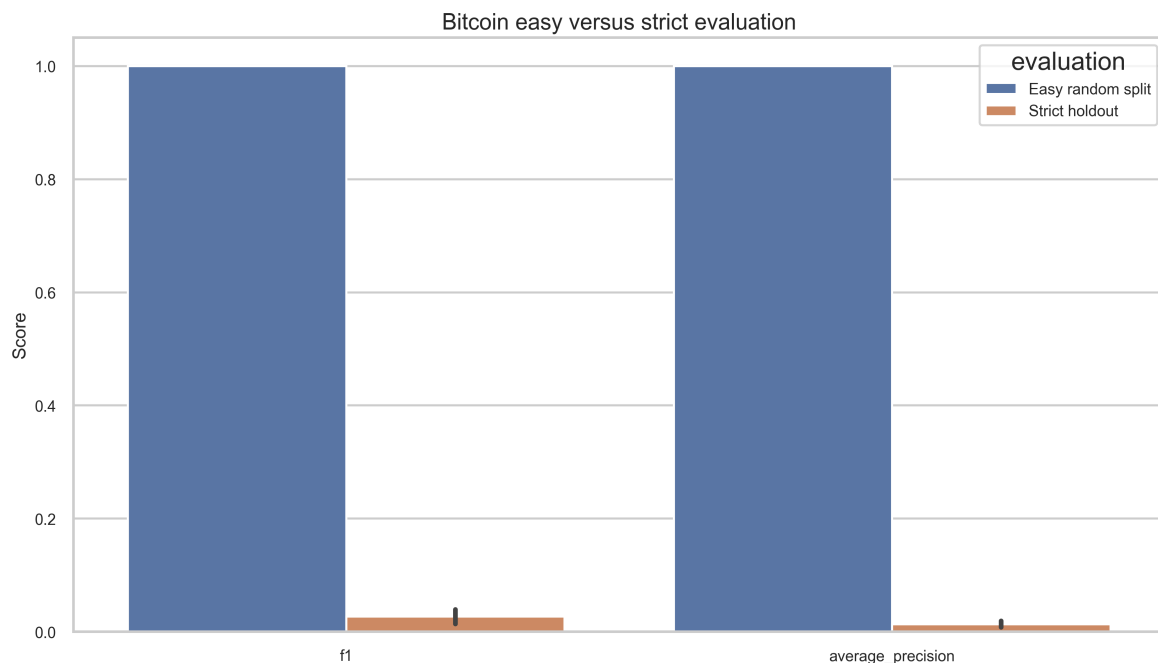


Figure 5.4: Bitcoin performance under the earlier easy random-split evaluation and the newer strict temporal-holdout evaluation.

Under the earlier easy path, the Bitcoin rows in the older combined results file reached effectively perfect metrics. In the strict report pipeline, by contrast, the best Bitcoin F1-score falls to 0.0393. This difference is too large to be dismissed as normal experimental variation. It shows that the protocol itself materially changed what the models appeared to achieve.

The strict report pipeline made four concrete methodological changes. First, it removed identity-like and temporal fields from the feature matrix. Second, it tuned only on years before the holdout year. Third, it used grouped cross-validation by year during tuning. Fourth, it reported final performance on an unseen 2017 holdout. Together, these changes substantially reduced optimism and produced a much more credible picture of generalisation.

5.7 Summary of Main Findings

The main results of this chapter can be summarised as follows.

First, the from-scratch implementations are broadly validated against the scikit-learn baselines. Exact parity is achieved for several model-dataset pairs, especially KNN and Decision Trees on Iris and KNN on Bank Marketing. The paired prediction analysis strengthens this further by showing near-parity on many of the standard benchmark pairs.

Second, no single algorithm family dominates every task. The sklearn SVM is strongest on Iris and Breast Cancer, whereas the custom Decision Tree is strongest on Bank Marketing and on the strict Bitcoin holdout.

Third, Bitcoin Heist remains by far the hardest task in the study under the credible strict pipeline. The strongest strict Bitcoin model is the custom Decision Tree, but even that reaches only $F1 = 0.0393$ and average precision = 0.0189.

Fourth, the Bitcoin paired analysis reveals that the SVM divergence is substantive rather than trivial. The custom and sklearn SVMs behave radically differently on the same strict holdout, with the custom model operating in a highly recall-oriented regime and the sklearn model remaining extremely conservative.

Fifth, the Bitcoin redesign remains the strongest methodological finding of the report. The earlier easy random-split path made Bitcoin appear almost perfectly solvable. The stricter report-era pipeline overturned that conclusion and showed that evaluation design materially changed the substantive interpretation of model performance.

These findings provide the basis for the discussion chapter, which interprets why these patterns emerged and what they imply for the trustworthy comparison of classical machine learning methods on imbalanced tabular data.

Chapter 6: Discussion

This chapter interprets the results presented in Chapter 5 and relates them back to the aims, research questions, and literature reviewed earlier in the report. The core argument that emerges is twofold. First, the from-scratch implementations are broadly valid and competitive on standard tabular benchmarks, especially in the case of k-Nearest Neighbours and Decision Trees, as shown in Table 5.1 and the standard benchmark results tables in Chapter 5. Second, the Bitcoin Heist experiments show that evaluation design can materially alter the apparent conclusion of a machine learning benchmark, most clearly illustrated by Figure 5.4 and the strict Bitcoin results in Table 5.6. In that sense, the project is not only a comparison of classical classifiers, but also a study in trustworthy evaluation under class imbalance and temporal structure.

The most important point is therefore not that one algorithm family won every dataset, because it did not. Rather, it is that model behaviour depended strongly on dataset structure, metric choice, and the validity of the evaluation protocol. This aligns closely with the literature reviewed in Chapter 2, which suggested that no universally best classifier should be expected [17, 18], that metric choice becomes central under class imbalance [14, 15, 21], and that weak evaluation design can produce misleadingly optimistic conclusions [13, 12, 11].

6.1 Interpretation of the Results

Taken as a whole, the results support the claim that the project successfully implemented three distinct supervised learning families from first principles and evaluated them within a coherent framework. The standard datasets show that the custom implementations are not merely running, but behaving in ways that are broadly consistent with trusted library baselines. Exact parity for custom and sklearn KNN on Iris, Bank Marketing, and Breast Cancer, together with exact parity for the custom and sklearn Decision Trees on Iris, as reported in Table 5.1, provides strong evidence that the core implementation logic is valid. The custom SVM is less tightly aligned to sklearn, but it remains competitive enough on Iris, Bank Marketing, and especially Breast Cancer to support the conclusion that it is a meaningful from-scratch model rather than a failed approximation.

At the same time, the project does not support any simplistic conclusion such as “SVM is best” or “Decision Trees dominate.” Instead, the pattern is more interesting. SVM is strongest on the cleaner and more margin-friendly datasets, particularly Breast Cancer, as shown in Table 5.3. Decision Tree is the most practically robust family across the full project, since it is exact on Iris, strongest on Bank Marketing, and also the best final model on strict Bitcoin, as shown in Tables 5.2, 5.4, and 5.6. KNN is the most stable parity case against sklearn, but it becomes less competitive on the harder and more heterogeneous tabular tasks. This is exactly the kind of dataset- and inductive-bias-dependent behaviour that the theory chapters suggested should emerge [2, 1, 3].

The most significant overall finding, however, is not attached to any one algorithm family. It is methodological. The Bitcoin Heist case shows that evaluation design can change the substantive conclusion of the entire experiment. Under the earlier easier path, Bitcoin appeared almost solved. Under the stricter report-era pipeline, the best Bitcoin F1-score fell to 0.0393, as shown by the contrast between Figure 5.4 and Table 5.6. That change is too large to be explained by normal random variation alone. It indicates that the earlier evaluation was optimistic and that the stricter design provides a much more credible estimate

of generalisation. In practical terms, the project therefore solves two problems at once. It benchmarks classifiers, but it also demonstrates why trustworthy evaluation matters as much as algorithm choice.

6.2 From-Scratch Models Versus Library Baselines

One of the main research questions concerned whether the custom models were sufficiently close to their scikit-learn counterparts to support meaningful comparison. The answer is broadly yes, but with important qualification.

For KNN, the evidence is especially strong. Exact parity against scikit-learn on multiple final holdouts, visible in Table 5.1 and reinforced by the paired agreement results in Table 5.7, suggests that the custom distance computation, neighbour selection, and voting logic are functionally correct. This is not a trivial achievement, because it shows that a relatively simple but carefully implemented from-scratch classifier can replicate trusted library behaviour on multiple datasets. Decision Tree is similarly strong overall. Exact parity on Iris, very small differences on Breast Cancer, and a modest but interpretable gap on Bank Marketing all support the view that the custom recursive tree is a credible implementation of classical tree induction rather than merely a toy approximation.

The SVM evidence is more mixed. This is not surprising. Of the three families, SVM is the most optimisation-sensitive, the most dependent on feature scaling, and the least straightforward to reproduce exactly from first principles. The custom solver uses a Pegasos-style primal stochastic method, while sklearn's `LinearSVC` relies on a different optimisation framework. Exact equivalence was therefore never a realistic expectation. What matters instead is whether the custom SVM remained competitive and behaved plausibly. On that standard, the results are still positive. The custom SVM is close to sklearn on Iris, competitive on Bank Marketing, and meaningfully strong on Breast Cancer, as shown in Tables 5.2, 5.4, and 5.3. It is therefore better understood as a credible but less tightly aligned implementation than KNN or Decision Tree.

The paired prediction analysis in Chapter 5 strengthens this conclusion. Most standard-dataset custom/sklearn pairs show high agreement rates in Table 5.7, and McNemar-style comparison in Table 5.8 suggests that only a small number of dataset-model pairs differ significantly on the shared final test sets. This should not be overinterpreted as proof of universal equivalence, but it does support a measured claim: the custom KNN and Decision Tree are strongly validated, while the custom SVM is competitive but less consistently matched to sklearn. That is a strong outcome for an undergraduate project and directly answers the implementation-correctness objective.

6.3 Impact of Class Imbalance and Metric Choice

The results also reinforce one of the central theoretical points from Chapter 2: in imbalanced classification, metric choice materially changes what the results appear to say [14, 15, 21]. This is most obvious on Bitcoin Heist, but it also matters for Bank Marketing and, to a lesser extent, Breast Cancer.

On Bank Marketing, the accuracy values for the main models are all quite close, yet the F1-scores separate them more clearly, as seen in Table 5.4. This matters because the positive

class is the one of real interest. A model can achieve high overall accuracy by leaning heavily toward the majority class, but that is less informative than understanding how well it balances recovering true subscribers against generating false alarms. The fact that the custom Decision Tree achieved the best Bank Marketing F1 therefore matters more than small differences in global accuracy.

Breast Cancer tells a slightly different but equally important story. Here, accuracy, F1, ROC-AUC, and average precision all point in broadly the same direction because the dataset is cleaner and the models perform strongly overall, as shown in Table 5.3. Even so, the clinically relevant interpretation still depends more on malignant-case recovery than on overall correctness. For that reason, the strong SVM performance is most convincing not merely because accuracy is high, but because F1 and ranking-based metrics are also high. This produces a much more reassuring picture of classification quality than accuracy alone would.

Bitcoin Heist is where the lesson becomes unavoidable. Accuracy is actively misleading if read in isolation. The sklearn SVM achieves extremely high accuracy while recovering almost none of the positive cases, whereas the custom SVM does the opposite, recovering almost all positives while collapsing in accuracy and precision, as reported in Table 5.6. Neither model can be judged properly using accuracy as the headline metric. Instead, precision, recall, F1, and average precision are the only metrics that convey the actual operational trade-off. This strongly validates the decision to use F_1 as the main tuning criterion and to treat ranking-based metrics as especially important for Bitcoin. More broadly, it shows that the real-world usefulness of a classifier depends not only on how often it is right overall, but on which mistakes it makes and in what proportions.

6.4 Dataset-Specific Behaviour

6.4.1 Iris

Iris functions mainly as a methodological control, and the results support that role well. The exact parity between the custom and sklearn KNN and Decision Tree models in Table 5.2 is plausible because the dataset is small, balanced, low-dimensional, and sufficiently clean that similar hyperparameters can easily lead to the same effective decision boundaries. The slight underperformance of the custom SVM relative to sklearn is not especially concerning. A cautious explanation is that the custom multiclass path, built on top of a binary Pegasos-style solver, is simply not identical to sklearn's implementation. The important point is not that the custom SVM failed, but that it remained close while the other two families matched exactly. This makes Iris a successful sanity-check benchmark rather than an area of concern.

6.4.2 Breast Cancer

Breast Cancer is the clearest case in which the SVM family benefits from the dataset structure. It is a binary, numerical, high-signal task and the final pipeline scales the features, all of which favour a large-margin model [3, 8]. The fact that the sklearn SVM is the strongest overall model on this dataset, as shown in Table 5.3, therefore fits theory well. The custom SVM also performs strongly, which supports the claim that the from-scratch large-margin implementation is meaningful on a clean binary problem.

KNN is also strong on this dataset, suggesting that local neighbourhood structure remains informative after preprocessing. Decision Tree is weaker, which is plausible because axis-

aligned recursive splits may lose some of the smoother separation captured by distance-based or margin-based methods. However, this dataset also illustrates the need for caution. The authoritative pipeline retains the identifier column, which means the final runs are not based on a perfectly clean 30-feature medical representation. This does not make the findings useless, but it does mean that this chapter should stop short of overclaiming medical realism or clinical readiness.

6.4.3 Bank Marketing

Bank Marketing is the clearest case in which Decision Tree emerges as the most practically robust family. The custom Decision Tree achieves the strongest final F1-score in Table 5.4, which fits the intuition that a split-based model can work well on a large, heterogeneous, one-hot encoded tabular dataset. Tree-based partitioning is often a natural fit for this type of structured commercial data [6, 5], and the results here support that view.

The custom SVM also performs credibly and slightly exceeds the sklearn SVM on F1. The safest explanation is not that the custom optimisation is universally better, but that the custom model's class-weighted decision behaviour on this specific split recovered more positives, while sklearn remained slightly more precise. That is consistent with the paired prediction results in Table 5.7 and is a much more defensible interpretation than any claim of absolute superiority.

KNN is weaker on Bank Marketing than on the smaller cleaner datasets, which is also plausible. One-hot encoded tabular spaces are less favourable for pure distance-based methods than low-dimensional numeric benchmarks. This fits both the literature on KNN's sensitivity to scale and dimensionality [3] and the broader project narrative that no single family is best everywhere.

Again, a caveat matters. The authoritative Bank Marketing pipeline retains `duration`, which is known to be highly predictive but may not be available in the same form at real decision time [25]. This means the Bank results are valid as benchmark results within the current pipeline, but should not be translated directly into strong deployment claims.

6.4.4 Bitcoin Heist

Bitcoin Heist is the most important dataset in the entire project, not because performance is highest, but because it most clearly exposes the difference between optimistic benchmarking and credible evaluation. Under the strict pipeline, Bitcoin becomes both the hardest dataset and the most informative methodological case study.

The best strict Bitcoin model is the custom Decision Tree. This should not be interpreted as evidence of strong practical detection performance. Its absolute scores remain low, as shown in Table 5.6. However, it does appear to provide the best balance among weak but non-trivial precision, recall, and F1. A cautious explanation is that a weighted tree can exploit non-linear structure in the reduced set of retained transaction features without collapsing into the extreme threshold behaviour seen in SVM. That makes Decision Tree the most practically robust Bitcoin family in this project, even though performance remains far from deployment-ready.

The SVM behaviour is much more revealing. The custom and sklearn SVMs do not simply differ in performance level. They behave in fundamentally different ways on the same strict holdout. The custom SVM becomes ultra-high-recall and ultra-low-precision, predicting the

positive class for most of the holdout set. The sklearn SVM becomes extremely conservative, predicting almost everything as negative. This is one of the strongest discussion points in the dissertation because it shows that even when two models belong to the same nominal family, their operating behaviour can diverge radically depending on optimisation, scoring, and thresholding. This divergence is visible numerically in Table 5.6, in paired form in Tables 5.7 and 5.8, and visually in the threshold-tradeoff plots in Figures 5.2 and 5.3.

A particularly important insight here is that the two SVMs are not being thresholded over directly comparable score spaces. The custom SVM produces sigmoid-transformed, probability-like scores, whereas sklearn’s `LinearSVC` produces raw decision margins. This means the models are not simply two versions of the same calibrated detector. They are being thresholded over fundamentally different output representations. That helps explain why the custom SVM classifies most holdout cases as positive while the sklearn SVM classifies almost none as positive. In other words, the divergence is not just about model capacity. It is also about score representation and threshold behaviour.

This point becomes even stronger when paired with the class-overlap analysis and the precision-recall evidence from Chapter 5. In the strict Bitcoin setting, the SVM score distributions for the positive and negative classes overlap heavily, and the PR evidence shows that the best strict models remain only modestly above the class-prevalence baseline. This explains the weak average precision and the instability of threshold-based behaviour. The discussion implication is important: under a credible temporal split, Bitcoin is genuinely hard, and the current feature set plus simple linear or axis-aligned models recover only limited usable signal. That is a much stronger academic result than pretending the task was solved.

6.5 Technical Lessons from Development

From a software-engineering perspective, the project demonstrates more than straightforward implementation. Several of the strongest outcomes came from debugging-driven redesign.

The clearest example is the Bitcoin methodology shift. The project did not simply report the initial near-perfect Bitcoin results and move on. Instead, those results were treated as suspicious, investigated, and ultimately replaced with a stricter report-era pipeline whose effect is made explicit in Figure 5.4. That pipeline used feature dropping, grouped cross-validation by year, threshold selection from pre-holdout CV predictions, and a strict unseen 2017 holdout. This is arguably the strongest evidence of technical maturity in the whole dissertation, because it shows that the project evolved in response to a real methodological problem rather than simply adding more code.

A second important refinement is KNN batching. The final implementation does not remain a fragile classroom prototype. It explicitly addresses distance-matrix scalability by processing queries in batches. That is a real engineering improvement and shows awareness of the gap between mathematical correctness and practical usability.

The SVM also shows meaningful development beyond a toy implementation. Class weighting, early stopping, averaged parameters, and threshold-aware evaluation all became important to the final story, especially in the Bitcoin analysis reflected in Tables 5.5, 5.6, and 5.8. Even where the custom SVM does not perfectly match sklearn, the implementation still reflects substantial technical work and supports useful analytical conclusions. The key point is that the project did not only produce benchmark numbers. It produced an experimentation framework that was refined, debugged, and made more rigorous over time.

6.6 Comparison with the Literature

Overall, the results align well with the literature reviewed in Chapter 2. First, they support the view that there is no universally best classifier [17, 18]. Different families perform best on different datasets, and even within the same family, behaviour can vary dramatically with evaluation design.

Second, the results support the claim that metric choice matters under imbalance [14, 15, 21]. This is visible on Bank Marketing and becomes unavoidable on Bitcoin, where accuracy alone would tell a highly misleading story, as demonstrated directly in Table 5.6. Third, the results support the literature on evaluation validity and leakage [13, 12, 11]. The Bitcoin case, especially as shown in Figure 5.4, is a direct demonstration that temporally invalid evaluation can produce inflated and potentially dangerous conclusions.

At the algorithm level, the findings are also broadly plausible relative to theory. KNN performs best on the smaller and cleaner datasets and is weaker on the larger more heterogeneous task, which fits its known sensitivity to scale and dimensionality [3]. Decision Tree is especially robust on structured tabular problems, which is consistent with classical tree literature [6, 5]. SVM is strongest on the clean scaled numerical dataset, Breast Cancer, which is consistent with the expectation that large-margin methods perform well when the feature space is already informative [3, 8]. The project therefore does not produce bizarre or literature-defying outcomes. Instead, it broadly confirms established expectations while adding a particularly strong cautionary lesson about Bitcoin evaluation.

6.7 Overall Discussion

The strongest overall conclusion from this chapter is that the project should be understood as evidence of both implementation competence and methodological competence. The standard datasets show that the custom models are broadly valid and often competitive with sklearn, as summarised in Figure 5.1, Table 5.1, and the paired comparison tables. The Bitcoin case shows that careful evaluation design can matter as much as the choice of classifier itself. Together, these findings answer the core research questions in a way that is stronger than a simple “winner” table.

At the same time, the discussion should remain measured. The project does not show universal superiority of any custom model, and it does not justify strong deployment claims for Bank Marketing, Breast Cancer, or Bitcoin. In particular, the strict Bitcoin results are better read as evidence of limited but non-trivial signal under credible evaluation, not as evidence that ransomware detection has been solved. Likewise, the retained Breast Cancer ID field and Bank Marketing `duration` feature mean that some of the strongest benchmark scores should be read with caution.

Within those boundaries, however, the project makes a meaningful contribution. It shows that from-scratch classical machine learning implementations can be made competitive, that their behaviour can be interpreted in light of inductive bias and dataset structure, and that apparently excellent results can collapse once evaluation becomes realistic. That final point is the most important. The project’s most valuable lesson is not simply which model performed best, but why trustworthy evaluation must be treated as part of the machine learning problem itself.

Chapter 7: Project Evaluation and Reflection

This chapter evaluates the project against its original aims, reflects on the main strengths and weaknesses of the final system, and identifies the most important lessons arising from its development. Unlike Chapter 6, which interpreted the experimental findings in relation to theory and application context, this chapter focuses on the quality of the project as a piece of technical and methodological work. The central judgement is that the project met its main objectives and produced a credible comparative framework, but also revealed important limitations in feature design, evaluation symmetry, and experimental realism.

7.1 Assessment Against Original Objectives

The project set out to implement and compare three classical supervised learning algorithms from first principles, namely k-Nearest Neighbours, Decision Trees, and Support Vector Machines, within a unified experimentation framework. On this objective, the project was successful. Custom versions of all three model families were implemented, integrated into a shared pipeline, and evaluated on multiple datasets under broadly matched conditions.

The project also aimed to build a reusable experimental framework covering preprocessing, cross-validation, hyperparameter search, metric computation, and result logging. This objective was also achieved. The final repository supports a coherent workflow from raw data to final tables and figures, and this was one of the strongest practical outcomes of the work. In particular, the shared pipeline made it possible to apply methodological corrections, especially in the Bitcoin experiments, without rewriting the entire codebase.

A further objective was to validate the custom models against scikit-learn baselines. This was achieved most clearly for KNN and Decision Trees, where exact or near-exact parity was observed on several standard datasets. The custom SVM was less tightly aligned to sklearn, but it remained competitive enough on the standard benchmarks to support the claim that it was a meaningful implementation rather than an unstable or incorrect one.

The project also aimed to compare model behaviour on imbalanced and cost-sensitive tabular problems. This objective was achieved, but with an important qualification. The comparison was strongest on Iris, Breast Cancer, and Bank Marketing. Bitcoin Heist ultimately contributed more as a methodological case study than as a strong predictive success. This should not be viewed as a failure of the project. On the contrary, the Bitcoin results became one of the most valuable outcomes because they exposed the importance of evaluation validity.

Overall, the project met its principal objectives. Its strongest achievement was not the discovery of a universally best classifier, but the creation of a from-scratch benchmarking framework that produced both algorithmic and methodological insight.

7.2 What Went Well

Several aspects of the project went particularly well.

First, the from-scratch implementations were strong enough to support meaningful compari-

son. KNN and Decision Tree were especially successful in this regard. Their close agreement with sklearn on multiple datasets gave the project a firm technical foundation and reduced the risk that later comparisons were driven mainly by implementation error.

Second, the software architecture worked well. The decision to separate model logic, evaluation, tuning, and plotting into reusable components proved valuable throughout the project. This improved debugging, maintainability, and experimental consistency. It also made the final system feel like a small research framework rather than a collection of disconnected scripts.

Third, the project became methodologically stronger over time. The most important example was the redesign of the Bitcoin pipeline after the early results appeared unrealistically strong. Rather than treating those results as a success and moving on, the evaluation protocol was tightened through feature removal, grouped cross-validation by year, and a temporal holdout. This change substantially improved the credibility of the final report and was probably the single strongest sign of technical maturity in the whole project.

Finally, the report itself now tells a coherent story. The literature review, implementation chapter, methodology, results, and discussion all support the same central argument: that algorithm comparison is only meaningful when evaluation design is trustworthy. That coherence is one of the strongest qualities of the final submission.

7.3 What Went Wrong

The project also encountered several genuine problems.

The most important issue was that some early experimental conclusions were too optimistic. This was most visible in Bitcoin Heist, where the earlier random-split path produced results that were not credible once the temporal structure of the dataset was taken seriously. This revealed that the initial evaluation design had been too weak for the task and had to be corrected later.

A second problem was imperfect feature handling in some final pipelines. The authoritative Breast Cancer runs retained the identifier column, and the authoritative Bank Marketing runs retained the `duration` feature. Neither issue invalidates the report, but both reduce the cleanliness of the final benchmark and impose limits on how strongly the results can be interpreted as realistic deployment evidence.

A third difficulty was uneven symmetry across model-dataset combinations. Not every family received exactly the same tuning depth under exactly the same conditions, especially on Bitcoin, where computational constraints limited some paths more than others. This means the comparative claims are stronger descriptively than they are as a perfectly controlled benchmark study.

A final weakness was that the project did not include a broader suite of formal statistical comparisons from the start. McNemar-style paired tests were later added for the final holdouts, which strengthened the analysis, but the inferential component is still more limited than would be expected in a larger-scale benchmarking study.

7.4 Lessons Learned

The most important lesson from the project is that trustworthy evaluation must be treated as part of the machine learning problem itself. It is not enough to implement a model correctly, tune it carefully, and report high metrics. If the evaluation protocol is weak, then the apparent success may say more about the benchmark than about the classifier. The Bitcoin case made this lesson unavoidably clear.

A second lesson is that implementing algorithms from first principles deepens understanding in a way that using library calls alone does not. Writing KNN, Decision Tree, and SVM manually made issues such as regularisation, optimisation behaviour, tie handling, batching, class weighting, and thresholding much more concrete. This was especially true for the SVM, where the distance between mathematical formulation and practical solver behaviour became much clearer through implementation.

A third lesson is that software design choices matter to research quality. The modular structure of the codebase was not only a programming convenience. It directly improved the quality of the experimental work by making debugging easier, reducing duplication, and allowing global methodological changes to be applied consistently. In that sense, good software engineering supported better science.

7.5 Limitations

The final project has several limitations that should be recognised clearly.

The first is that some benchmark pipelines retained features that weaken real-world interpretability. The retained Breast Cancer identifier field and Bank Marketing `duration` feature are the clearest examples. This means that some of the reported scores should be read as benchmark results within the current pipeline rather than as clean deployment-ready estimates.

The second is that the Bitcoin feature set and model families were not strong enough to produce practically strong detection under the strict temporal protocol. The final results show limited but non-zero signal, not a reliable ransomware detector. This is an important boundary on the claims of the project.

The third is that the project remains primarily a descriptive comparative study rather than a fully inferential benchmark. Although paired prediction-level comparison was included in the final analysis, the study does not attempt a more extensive statistical ranking procedure across all datasets and model families.

The fourth is that reproducibility is stronger at the level of scripts and saved outputs than at the level of portable environment specification. The repository supports rerunning the experiments, but dependency management could be stronger in a more mature research package.

7.6 Future Work

There are several clear directions for future work.

The first is to strengthen dataset realism by cleaning the remaining feature-handling issues, especially removing the Breast Cancer identifier field and reconsidering the role of **duration** in Bank Marketing. This would make the final benchmark cleaner and the practical interpretation of the results stronger.

The second is to extend the Bitcoin work with richer feature engineering and more realistic temporal modelling. The strict pipeline showed that the current feature set makes the task genuinely difficult. Future work could explore calibrated models, richer temporal representations, or ensemble methods better suited to severe imbalance and distribution shift.

The third is to improve the inferential strength of the comparison by adding broader statistical testing and confidence intervals across repeated evaluation settings. This would strengthen the project as a formal benchmark study.

The fourth is to expand the framework itself. The current system could be extended to include additional baselines such as logistic regression or random forests, more automated experiment configuration, and stronger environment reproducibility. These changes would not alter the main conclusions of the present project, but they would make the framework more useful for future comparative work.

Overall, the project provides a solid foundation for that future development. Its strongest legacy is not a single best-performing model, but a clearer understanding of how classical machine learning systems should be implemented, evaluated, and interpreted when the data are imbalanced and the cost of overclaiming is high.

Chapter 8: Conclusion

8.1 Final Conclusion

This report set out to determine how three classical supervised machine learning algorithms, namely k-Nearest Neighbours, Decision Trees, and Support Vector Machines, could be implemented from scratch, validated against trusted library baselines, and compared fairly on tabular classification problems in which class imbalance and asymmetric error costs materially affect what should count as good performance. The project was motivated by the observation that many practically important decision problems in banking, medicine, and cybersecurity are not simply classification tasks in the abstract, but imbalanced and cost-sensitive problems in which minority-class performance matters far more than headline accuracy alone. For that reason, the report was never intended to identify a universally superior classifier. Its purpose was to build a rigorous experimental framework and to use that framework to draw trustworthy conclusions about algorithm behaviour under realistic conditions.

Taken as a whole, the project achieved that aim. The from-scratch implementations of k-NN, Decision Trees, and a Pegasos-based linear SVM were successfully developed within a unified experimentation framework that supported shared preprocessing, cross-validation, hyperparameter search, metric computation, comparison against scikit-learn baselines, and reproducible output generation. Across the standard benchmark datasets, the custom models were broadly validated. Exact parity with scikit-learn was achieved in several cases, especially for k-NN and Decision Trees, while the custom SVM remained competitive even where it did not match the library implementation exactly. This is an important outcome because it shows that the project produced more than a collection of runnable scripts. It produced technically credible from-scratch implementations whose behaviour was sufficiently faithful to support meaningful comparison and analysis.

The empirical results also showed clearly that model performance depended strongly on dataset structure. On Iris, the custom models behaved as expected on a small, balanced sanity-check benchmark, providing evidence that the basic implementation and evaluation pipeline were functioning correctly. On Breast Cancer, the SVM family performed most strongly, which is consistent with the view that large-margin methods are well suited to clean, scaled, numerical binary classification problems. On Bank Marketing, the custom Decision Tree emerged as the strongest model by final F_1 -score, suggesting that flexible split-based models can be especially effective on larger heterogeneous tabular data. KNN, while stable and well validated against scikit-learn, proved less competitive on the more complex and higher-dimensional tasks. These findings support the central theoretical expectation developed earlier in the report: there is no universally best classifier, because performance depends on the relationship between inductive bias, feature representation, class imbalance, and task structure.

However, the most important contribution of the project lies not in any single model ranking. It lies in the Bitcoin Heist case study, which showed that evaluation design can materially alter the apparent conclusion of a machine learning benchmark. Under the earlier easier evaluation path, Bitcoin appeared almost perfectly solvable. Under the stricter report-era pipeline, which removed problematic fields, tuned on pre-holdout years only, used grouped cross-validation by year, and evaluated final performance on an unseen 2017 temporal hold-out, the best F_1 -score fell dramatically to 0.0393. This change was not a minor statistical fluctuation. It fundamentally changed the meaning of the result. What had initially looked like a highly successful ransomware detection benchmark became, under more credible evalu-

ation, a genuinely difficult rare-event detection problem with only limited recoverable signal.

That methodological shift is the strongest intellectual result in the dissertation. It shows that apparently excellent model performance can be an artefact of optimistic evaluation rather than evidence of genuine generalisation. In that sense, the project makes a broader point that extends beyond the three selected algorithm families. Trustworthy machine learning depends not only on choosing a model, but on designing an evaluation protocol that reflects the real structure of the deployment problem. This is especially important in temporally structured and imbalanced settings, where naive random splits can make a difficult task look artificially easy. The Bitcoin experiments therefore transformed the project from a straightforward classifier comparison into a more mature study of methodological validity.

The project also reinforced the importance of metric choice. Across the imbalanced datasets, accuracy alone was often an inadequate and at times actively misleading summary of performance. This was most obvious on Bitcoin Heist, where one model could achieve very high accuracy by predicting almost everything as negative, while another could achieve very high recall by predicting almost everything as positive. Neither behaviour could be understood properly without examining precision, recall, F_1 , average precision, and threshold-dependent operating behaviour. More broadly, the results confirmed that model usefulness must be judged in relation to the kinds of errors made and the practical cost of those errors, not merely by overall correctness.

The report should nevertheless be read within its limits. The project does not establish the universal superiority of any model family, nor does it justify strong deployment claims in banking, medicine, or cybersecurity. Some pipelines retained features such as the Breast Cancer identifier field and the Bank Marketing duration feature, both of which require caution when interpreting real-world realism. The strict Bitcoin results, although methodologically more credible, also show that the current feature set and model families recover only limited useful signal under temporal holdout. These constraints matter, and they define the boundary of the report's claims. Even so, they do not weaken the central achievement. On the contrary, the willingness to surface them openly is part of what makes the conclusions more credible.

Overall, this project made a meaningful contribution at two levels. At a technical level, it demonstrated that classical machine learning methods can be implemented from first principles, validated systematically, and benchmarked within a coherent experimental framework. At a methodological level, it showed that careful evaluation is not secondary to machine learning practice, but central to it. The final conclusion of the report is therefore clear. Classical machine learning algorithms can behave competitively and predictably on tabular classification problems when implemented carefully, but the most important determinant of trustworthy conclusions is not simply which algorithm is chosen. It is whether the evaluation process itself is rigorous enough to deserve trust. That is the most important lesson of the project, and the one with the greatest practical and academic value.

Bibliography

- [1] T. Hastie, R. Tibshirani, and J. Friedman. *The Elements of Statistical Learning*, 2nd ed. Springer, New York, 2009. DOI: 10.1007/978-0-387-84858-7.
- [2] T. M. Mitchell. *Machine Learning*. McGraw-Hill, New York, 1997.
- [3] C. M. Bishop. *Pattern Recognition and Machine Learning*. Springer, New York, 2006.
- [4] T. M. Cover and P. E. Hart. Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*, 13(1):21–27, 1967. DOI: 10.1109/TIT.1967.1053964.
- [5] J. R. Quinlan. Induction of decision trees. *Machine Learning*, 1(1):81–106, 1986. DOI: 10.1007/BF00116251.
- [6] L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone. *Classification and Regression Trees*. Wadsworth, Belmont, CA, 1984.
- [7] C. Cortes and V. Vapnik. Support-vector networks. *Machine Learning*, 20(3):273–297, 1995. DOI: 10.1007/BF00994018.
- [8] N. Cristianini and J. Shawe-Taylor. *An Introduction to Support Vector Machines and Other Kernel-Based Learning Methods*. Cambridge University Press, Cambridge, 2000. DOI: 10.1017/CBO9780511801389.
- [9] S. Shalev-Shwartz, Y. Singer, N. Srebro, and A. Cotter. Pegasos: primal estimated sub-gradient solver for SVM. *Mathematical Programming*, 127(1):3–30, 2011. DOI: 10.1007/s10107-010-0420-4.
- [10] R. Kohavi. A study of cross-validation and bootstrap for accuracy estimation and model selection. In *Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI)*, pages 1137–1145, 1995.
- [11] S. Arlot and A. Celisse. A survey of cross-validation procedures for model selection. *Statistics Surveys*, 4:40–79, 2010. DOI: 10.1214/09-SS054.
- [12] G. C. Cawley and N. L. C. Talbot. On over-fitting in model selection and subsequent selection bias in performance evaluation. *Journal of Machine Learning Research*, 11:2079–2107, 2010.
- [13] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman. Leakage in data mining: formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data*, 6(4):15:1–15:21, 2012. DOI: 10.1145/2382577.2382579.
- [14] T. Fawcett. An introduction to ROC analysis. *Pattern Recognition Letters*, 27(8):861–874, 2006. DOI: 10.1016/j.patrec.2005.10.010.
- [15] J. Davis and M. Goadrich. The relationship between precision-recall and ROC curves. In *Proceedings of the 23rd International Conference on Machine Learning (ICML)*, pages 233–240, 2006. DOI: 10.1145/1143844.1143874.
- [16] J. Demšar. Statistical comparisons of classifiers over multiple data sets. *Journal of Machine Learning Research*, 7:1–30, 2006.
- [17] R. Caruana and A. Niculescu-Mizil. An empirical comparison of supervised learning algorithms. In *Proceedings of the 23rd International Conference on Machine Learning (ICML)*, pages 161–168, 2006. DOI: 10.1145/1143844.1143865.

- [18] M. Fernández-Delgado, E. Cernadas, S. Barro, and D. Amorim. Do we need hundreds of classifiers to solve real world classification problems? *Journal of Machine Learning Research*, 15(1):3133–3181, 2014.
- [19] H. He and E. A. Garcia. Learning from imbalanced data. *IEEE Transactions on Knowledge and Data Engineering*, 21(9):1263–1284, 2009. DOI: 10.1109/TKDE.2008.239.
- [20] N. Japkowicz and S. Stephen. The class imbalance problem: A systematic study. *Intelligent Data Analysis*, 6(5):429–449, 2002. DOI: 10.3233/IDA-2002-6504.
- [21] T. Saito and M. Rehmsmeier. The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLOS ONE*, 10(3):e0118432, 2015. DOI: 10.1371/journal.pone.0118432.
- [22] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer. SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16:321–357, 2002. DOI: 10.1613/jair.953.
- [23] Scikit-learn developers. Model evaluation: quantifying the quality of predictions. *scikit-learn User Guide*. https://scikit-learn.org/stable/modules/model_evaluation.html Accessed: 10 April 2026.
- [24] A. Géron. *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow*, 3rd ed. O’Reilly Media, Sebastopol, CA, 2023.
- [25] S. Moro, P. Cortez, and P. Rita. A data-driven approach to predict the success of bank telemarketing. *Decision Support Systems*, 62:22–31, 2014. DOI: 10.1016/j.dss.2014.03.001.
- [26] S. Moro, P. Rita, and P. Cortez. *Bank Marketing* [Dataset]. UCI Machine Learning Repository, 2014. DOI: 10.24432/C5K306.
- [27] C. G. Akcora, Y. Li, Y. R. Gel, and M. Kantarcioglu. BitcoinHeist: Topological data analysis for ransomware prediction on the Bitcoin blockchain. In *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence (IJCAI-20)*, pages 4439–4445, 2020. DOI: 10.24963/ijcai.2020/612.
- [28] R. A. Fisher. The use of multiple measurements in taxonomic problems. *Annals of Eugenics*, 7(2):179–188, 1936. DOI: 10.1111/j.1469-1809.1936.tb02137.x.
- [29] Association for Computing Machinery. *ACM Code of Ethics and Professional Conduct*. ACM, 2018. <https://www.acm.org/code-of-ethics> Accessed: 10 April 2026.
- [30] British Computer Society. *BCS Code of Conduct*. BCS, The Chartered Institute for IT. <https://www.bcs.org/membership-and-registrations/become-a-member/bcs-code-of-conduct/> Accessed: 10 April 2026.
- [31] Information Commissioner’s Office. *Guidance on AI and data protection*. ICO, 2023. <https://ico.org.uk/for-organisations/uk-gdpr-guidance-and-resources/artificial-intelligence/guidance-on-ai-and-data-protection/> Accessed: 10 April 2026.
- [32] Information Commissioner’s Office. *Explaining decisions made with AI*. ICO. <https://ico.org.uk/for-organisations/uk-gdpr-guidance-and-resources/artificial-intelligence/explaining-decisions-made-with-artificial-intelligence/> Accessed: 10 April 2026.

- [33] National Cyber Security Centre. *Guidelines for secure AI system development*. NCSC, 2023. <https://www.ncsc.gov.uk/collection/guidelines-secure-ai-system-development> Accessed: 10 April 2026.
- [34] S. Barocas, M. Hardt, and A. Narayanan. *Fairness and Machine Learning: Limitations and Opportunities*. MIT Press, Cambridge, MA, 2023.
- [35] Europol. *Internet Organised Crime Threat Assessment (IOCTA) 2024*. European Union Agency for Law Enforcement Cooperation, 2024. <https://www.europol.europa.eu/publication-events/main-reports/internet-organised-crime-threat-assessment-iocta-2024> Accessed: 10 April 2026.

Appendix A: **Video Demonstration Link**

A short video demonstration of the project is available at:

<https://youtu.be/B-qYwy8o8LI?si=pW-GZe0DE4TzkgQy>

Appendix B: User Manual

This software is a Python-based machine learning benchmarking framework for comparing from-scratch implementations of k-Nearest Neighbours, Decision Trees, and a Pegasos-style linear Support Vector Machine with matched `scikit-learn` baselines.

The main code is stored in `src/`. Model implementations are in `src/algorithms/`, shared preprocessing and evaluation code in `src/evaluation/`, experiment scripts in `src/scripts/experiments/`, reporting scripts in `src/scripts/reporting/`, datasets in `src/data/raw/`, and final outputs in `src/results/`.

The main benchmark workflow is run with `scripts/experiments/run_full_pipeline.py`.

The authoritative non-Bitcoin final comparison is saved as `results/final_evaluation_all_models_strict.csv`.

The authoritative Bitcoin workflow uses `run_bitcoin_grid_report.py` and `run_bitcoin_final_report_evaluation.py` with final metrics saved as `results/bitcoin_report_final_metrics.csv`.

All main scripts should be run from inside `src/`.

Appendix C: Installation Manual

The project requires Python 3.10 or later and the packages listed in `requirements.txt`: `numpy`, `pandas`, `scipy`, `scikit-learn`, `matplotlib`, `seaborn`, `plotly`, and `pytest`.

Installation from a clean environment is:

```
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
cd src
```

To reproduce the main benchmark outputs:

```
$env:FAST_PIPELINE="0"
python .\scripts\experiments\run_full_pipeline.py
```

To reproduce the strict Bitcoin report outputs:

```
python .\scripts\experiments\run_bitcoin_grid_report.py
python .\scripts\experiments\run_bitcoin_final_report_eval.py
python .\scripts\reporting\analyse_bitcoin_results.py
python .\scripts\reporting\plot_report_figures.py
```

The datasets are expected to be present under `src/data/raw/`. A common issue is running scripts from the repository root instead of `src/`.